Function Calling in Large Language Models: Industrial Practices, Challenges, and Future Directions

MAOLIN WANG, City University of Hong Kong, Hong Kong, Hong Kong
YINGYI ZHANG, City University of Hong Kong, Hong Kong, Hong Kong
BOWEN YU, City University of Hong Kong, Hong Kong, Hong Kong
BINGGUANG HAO, Ant Group, Hangzhou, China
CUNYIN PENG, Ant Group, Hangzhou, China
YICHENG CHEN, Ant Group, Hangzhou, China
WEI ZHOU, Ant Group, Hangzhou, China
JINJIE GU, Ant Group, Hangzhou, China
CHENYI ZHUANG, Ant Group, Hangzhou, China
RUOCHENG GUO, Unaffliated, Hong Kong, Hong Kong
WANYU WANG, City University of Hong Kong, Hong Kong, Hong Kong
XIANGYU ZHAO, City University of Hong Kong, Hong Kong, Hong Kong

The swift evolution of Large Language Models (LLMs) like the GPT family, LLaMA, ChatGLM, and Qwen represents significant progress in artificial intelligence research. Despite their remarkable capabilities in generating content, these models encounter substantial challenges when producing structured outputs and engaging in dynamic interactions, particularly when they need to retrieve external information in real time. To address these limitations, researchers have developed the "Function Calling" paradigm. This approach enables language models to analyze user inquiries and engage with defined functions, thereby facilitating precise responses through connections to external sources, including databases, programming interfaces, and live data streams. This functionality has been successfully implemented across numerous sectors such as finance analytics, healthcare systems, and service operations. The implementation of function calling comprises

Maolin Wang, Yingyi Zhang, and Bowen Yu contributed equally to this research.

This research was partially supported by National Natural Science Foundation of China (No. 62502404), Hong Kong Research Grants Council (Research Impact Fund No. R1015-23, Collaborative Research Fund No. C1043-24GF, General Research Fund No. 11218325), Institute of Digital Medicine of City University of Hong Kong (No. 9229503), Ant Group (CCF-Ant Research Fund, Ant Group Research Fund).

Authors' Contact Information: Maolin Wang, City University of Hong Kong, Hong Kong, Hong Kong; e-mail: morin.wang@my.cityu.edu.hk; Yingyi Zhang, City University of Hong Kong, Hong Kong, Hong Kong; e-mail: yzhang6375-c@my.cityu.edu.hk; Bowen Yu, City University of Hong Kong, Hong Kong, Hong Kong; e-mail: bowyu2-c@my.cityu.edu.hk; Bingguang Hao, Ant Group Co Ltd, Hangzhou, Zhejiang, China; e-mail: bingguanghao7@gmail.com; Cunyin Peng, Ant Group Co Ltd, Hangzhou, Zhejiang, China; e-mail: pengcunyin@gmail.com; Yicheng Chen, Ant Group Co Ltd, Hangzhou, Zhejiang, China; e-mail: chenggejiayou@gmail.com; Wei Zhou, Ant Group Co Ltd, Hangzhou, Zhejiang, China; e-mail: fayi.zw@antgroup.com; Jinjie Gu, Ant Group Co Ltd, Hangzhou, Zhejiang, China; e-mail: jinjie.gujj@antgroup.com; Chenyi Zhuang (corresponding author), Ant Group Co Ltd, Hangzhou, Zhejiang, China; e-mail: chenyi.zcy@antgroup.com; Ruocheng Guo, Unaffliated, Hong Kong, Hong Kong; e-mail: rguo.asu@gmail.com; Wanyu Wang, City University of Hong Kong, Hong Kong, Hong Kong; e-mail: wanyuwang4-c@my.cityu.edu.hk; Xiangyu Zhao (corresponding author), City University of Hong Kong, Hong Kong, Hong Kong; e-mail: xy.zhao@cityu.edu.hk.

ACM 0360-0300/2026/02-ART239
https://doi.org/10.1145/3788284

three essential phases: preparation, execution, and processing. The preparation phase encompasses query analysis and function identification. During execution, the system evaluates whether a function is necessary, extracts relevant parameters, and oversees the operation. The processing phase concentrates on analyzing outcomes and crafting appropriate responses. Each phase presents unique difficulties, ranging from accurately selecting functions to managing complex parameter extraction and ensuring reliable execution. Researchers have established various evaluation frameworks and metrics to assess function calling performance, including success rates, computational efficiency, parameter extraction accuracy, and response quality indicators such as ROUGE-L evaluation scores. This survey systematically reviews the current landscape of function calling in LLMs, analyzing technical challenges, examining existing solutions, and discussing evaluation methodologies. We particularly focus on practical implementations and industrial applications, providing insights into both current achievements and future directions in this rapidly evolving field. For a comprehensive collection of related research papers and the Appendix file, please refer to our repository at GitHub.[1]

CCS Concepts: • **Computing methodologies** → **Neural networks**; **Natural language processing**; • **Information systems** → **Specialized information retrieval;**

Additional Key Words and Phrases: Large language models, function calling, industrial perspective, LLM agent

ACM Reference Format:
Maolin Wang, Yingyi Zhang, Bowen Yu, Bingguang Hao, Cunyin Peng, Yicheng Chen, Wei Zhou, Jinjie Gu, Chenyi Zhuang, Ruocheng Guo, Wanyu Wang, and Xiangyu Zhao. 2026. Function Calling in Large Language Models: Industrial Practices, Challenges, and Future Directions. *ACM Comput. Surv.* 58, 9, Article 239 (February 2026), 37 pages. https://doi.org/10.1145/3788284

1 Introduction

Recent advancements in artificial intelligence have ushered in a transformative era with the development of **large language models** (**LLMs**) such as GPT series, LLama [145], ChatGLM [18, 185] and Qwen [5]. These models, particularly effective in generation tasks due to their robust generative capabilities, have established themselves as vital for complex language tasks where traditional methods struggle. However, despite these advancements, LLMs face significant challenges in specific applications, particularly in generating highly structured outputs and engaging in timely real-world interactions [111, 122]. Even with evolving technology, LLMs often generate incorrect or outdated results for scenarios outside their training scope [21, 44], indicating certain limitations. For example, asking a LLM to analyze the latest stock market prices would be ineffective as it cannot access or retrieve real-time market data. This illustrates the model's limitation in handling dynamic information that changes frequently.

To overcome these deficiencies and more effectively leverage the potential of LLMs in enterprise and technical domains, the concept of "Function Calling" has been introduced [50]. In this context, a **Function** is defined as a predefined block of code, software, or tools that accept parameters and return a result. **The term Function Calling in the context of Large Language Models describes their proficiency at interpreting user requests and employing designated computational procedures to deliver precise, contextually suitable answers**. This technological advancement substantially improves these AI systems' accuracy when processing sophisticated inquiries while facilitating direct connections with numerous external resources–including data repositories, streaming information feeds, external application interfaces, alternative sophisticated language models, and additional services. To illustrate, these systems can acquire current weather conditions through meteorological service integrations and implement programming fragments using flexible code interpretation mechanisms. Additionally, this capability supports instantaneous analytical

[1] https://github.com/Applied-Machine-Learning-Lab/Awesome-Function-Callings

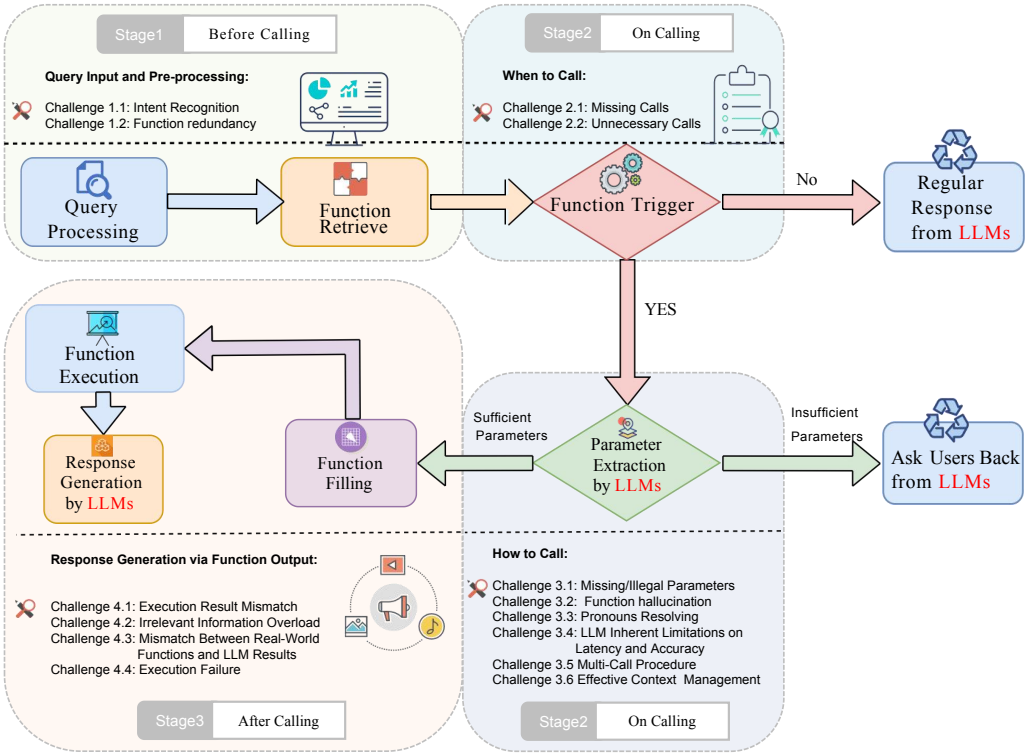

Fig. 1. Typical function calling pipeline and associated challenges. The pipeline consists of three main stages: pre-call (including query processing and function retrieval), on-call (covering function triggering and parameter handling), and post-call (involving function execution and response generation). Each stage faces distinct challenges ranging from intent recognition and function redundancy in the pre-call stage to parameter extraction and multi-call procedures during execution to result in mismatch and execution failures in the post-call stage. LLMs are typically utilized in Stage 2 and Stage 3.

processes and operational decisions crucial for various professional domains including financial research [96], medical information systems [41], and tailored support solutions [42, 48]. As a result, this innovation fundamentally transforms the architecture of automated intelligence platforms and their engagement patterns with human operators. For example, Wolfram Alpha's [42] plugin for ChatGPT leverages function calling to enable complex computational queries and data retrieval. Users can ask detailed mathematical, scientific, or statistical questions, and LLMs call Wolfram Alpha's API to provide accurate and up-to-date answers directly within ChatGPT. This significantly enhances the ability of LLMs to solve specific mathematical problems.

Thus, a significant ongoing task is to enhance LLMs, which are primarily trained in natural language, to develop robust capabilities for specialized function calling. As shown in Figure 1, for a comprehensive LLM function calling process, the entire process can be divided into three stages based on the timing of the call: pre-call, on-call, and post-call. In the pre-call stage, the model must pre-process the user's query, including planning and decomposing tasks and selecting suitable function candidates from a function pool. Next, when entering the on-call stage, the model needs to assess whether a function call is necessary to complete the current user's task, involving the correct triggering of a single function or multi-functions. This is because not every user query requires a function call; sometimes, standard dialogue suffices. The challenge in this stage is avoiding

unnecessary function calls and ensuring that needed function calls occur at the right time. Once a function call is triggered, the model first extracts parameters, identifying specific parameters from the user's query and incorporating them into the function. Common practices include matching and copying, but user language often contains a large amount of pronouns and unstructured expressions. For instance, asking about "today" involves a date that changes daily. To address this, we can use adapters or automatic rewriting in an intermediate language to optimize the process. Another major challenge in function calling is that if the required parameter volume is not provided, the model might need to ask the users to obtain the necessary information. Once all necessary parameters are prepared, the LLM will send the output results for the next step of processing. In the post-call stage, since the LLM output is in a natural language format and differs from real-world functions, the LLM output generated in natural language needs to be mapped to executable functions in the real physical world before being sent to the server for execution. During the execution stage, natural challenges include handling function execution failures and discrepancies between the function's results and the query.

Nonetheless, evaluating the effectiveness of function calling involves various strategies [181] and various aspects [180]. Metrics such as pass rate and win rate [109], average time per call [131], parameter identification ratio [181], retrieval metrics (such as Recall@K, NDCG@K, COMP@K) [110] and ROUGE-L score [67] provide a comprehensive assessment of the LLM's performance in executing function calls. Each of these metrics provides valuable insights into different aspects of the function calling process, but a holistic evaluation requires considering all these factors together to fully understand the performance and limitations of LLMs in real-world applications. Numerous studies, including API-BLEND [6], Seal-Tools [164], ToolBench2 [109], APIBank [59], ToolEyes [180], Rotbench [181], and others, have conducted extensive evaluations of the function calling capabilities of LLMs, encompassing a wide range of aspects.

In this article, we make several key contributions to the field of LLMs function calling:

- We clearly define the concept of function calling and provide a comprehensive function calling pipeline.
- We describe in detail the potential challenges encountered at each step of the function calling pipeline.
- We outline strategies for sample construction and training as well as model deployment to tackle all identified challenges. Importantly, we validate several of these strategies through carefully designed experiments, demonstrating their effectiveness.
- We discuss comprehensive evaluation strategies for function calling and metrics.
- We also summarize the main datasets related to function calling in LLMs.
- We discuss future challenges and outlooks in this field.

In this article, we present a comprehensive taxonomy of function calling in LLMs, which systematically categorizes key technical components and methodologies across the full lifecycle, from training data construction and model fine-tuning to deployment strategies and evaluation frameworks, providing a structured overview of both academic research advances and industrial applications in this rapidly evolving field. We have organized our survey article into the following sections: Section 3 explores the motivations behind function calling and analyzes it through the function calling workflow, highlighting the associated challenges. In Section 4, we detail strategies from the perspectives of sample construction and model fine-tuning that are designed to overcome specific challenges in function calling. Subsequently, Section 5 delves into strategies for tailored system deployment and model inference to tackle particular challenges associated with function calling. Then, Section 6 discusses potential future developments and open issues.

Additionally, we also provide supplementary information in the Appendix. **Appendix Section A** presents methods for assessing the performance of function calling. **Appendix Section B** examines existing industrial products along with essential datasets and benchmarks. Concise summaries of industrial data collection and staged state assessment are provided in **Appendix Section C**.

2 Related Works

Recent advances in LLMs have increasingly driven research into extending their functionality through external tools and function-calling mechanisms. Numerous survey papers have investigated various facets of this rapidly advancing field, surveying the topic from diverse perspectives–ranging from implicit function use, to workflow and agent studies where tools are only partially addressed.

Implicit function use surveys, including those by Zhao et al. [195] and Huang et al. [40] examined general LLM capabilities and planning mechanisms yet did not concentrate specifically on function calling or tool utilization. Likewise, reasoning-oriented surveys from Qiao et al. [107] and Sun et al. [139] did not address the integration of tools and workflows for complex task resolution. Research on retrieval-augmented generation from Gao et al. [26] and Zhao et al. [194] delivered comprehensive examinations of retrieval mechanisms but maintained primary focus on retrieval rather than encompassing tool learning frameworks.

In workflow surveys related to function calling, Mialon et al. [81] address foundational definitions and workflows, concentrating on how tools enhance reasoning and problem-solving capabilities in LLMs. Nevertheless, their research does not investigate the practical implementation challenges of function calling systems, including latency management, tool selection errors, or system failures. Qu et al. [111] present a comprehensive framework for tool learning, encompassing stages including task planning, tool selection, invocation, and response generation, yet do not examine optimization for real-world multi-task scenarios essential for industrial applications. Wang (2024 B) et al. [160] classify tool learning techniques and evaluation metrics but omit discussion of strategies for managing dynamic multi-tool environments or large-scale deployment.

Multiple surveys examine LLM-based autonomous agents. Xi et al. [169] and Wang (2024 A) et al. [152] acknowledge the significance of tools for agents yet lack systematic examination of invocation challenges and practical implementation specifics. Wang et al. [161] emphasize human-feedback-driven tool invocation but do not furnish solutions for practical challenges including invocation failures or latency in real-world deployments. Yang et al. [177] and Qin et al. [108] constitute more comprehensive endeavors. Yang et al. stress the integration of planning with tool invocation in complex tasks and examine deployment considerations, although their investigation remains predominantly methodological.

Table 1 presents a systematic comparison of existing surveys across six essential dimensions: definition/workflow, industrial challenges, sample construction, deployment and inference, evaluation, and futures and outlook. Although prior works have contributed significantly to understanding function calling and tool learning, limited studies comprehensively address all these dimensions concurrently.

Extending from these foundational contributions, our survey seeks to address critical gaps by delivering comprehensive treatment across all dimensions. We systematically examine industrial challenges encountered in real-world deployments, elaborate on sample construction strategies for training effective function calling models, explore deployment and inference optimizations, present comprehensive evaluation methodologies, and investigate future directions. Significantly, we validate multiple proposed strategies through meticulously designed experiments, demonstrating their practical effectiveness. This comprehensive methodology distinguishes our work from preceding surveys and furnishes both researchers and practitioners with actionable insights for developing and deploying function calling systems.

Table 1. Evaluation of Survey Papers Across Different Dimensions

Survey Paper	Definition or Workflow	Industrial Challenges	Sample Construction	Deployment & Inference	Evaluation	Futures & Outlook
Zhao et al. [194]	✓	✗	✗	✗	✓	✓
Gao et al. [26]	✓	✗	✗	✗	✓	✗
Xi et al. [169]	✓	✗	✗	✗	✓	✓
Sun et al. [139]	✓	✗	✗	✗	✓	✗
Qiao et al. [107]	✓	✗	✗	✗	✓	✗
Huang et al. [40]	✓	✗	✗	✗	✓	✗
Zhao et al. [195]	✓	✗	✗	✗	✓	✗
Wang (2023) et al. [161]	✓	✗	✓	✗	✓	✓
Wang (2024 A) et al. [152]	✓	✗	✗	✗	✓	✓
Wang (2024 B) et al. [160]	✓	✗	✗	✗	✓	✓
Qin et al. [108]	✓	✓	✓	✗	✓	✓
Yang et al. [177]	✓	✓	✓	✓	✗	✓
Mialon et al. [81]	✓	✗	✗	✗	✓	✗
Qu et al. [111]	✓	✗	✓	✗	✓	✓
Ours	✓	✓	✓	✓	✓	✓

3 Constructing the Foundation: Function Calling Pipeline and Challenges

We systematically analyze the function calling pipeline and its associated challenges in Section 3, providing a comprehensive framework for understanding this crucial aspect of LLM capabilities. This section presents a structured classification framework for identifying difficulties encountered throughout the function calling process. Our taxonomy systematically organizes these issues across multiple operational stages–beginning with query understanding prior to function invocation and extending through response management following execution completion.

3.1 Motivation

The incorporation of function calling mechanisms within Large Language Models serves to address inherent constraints these systems face beyond textual production capabilities [8]. Despite their sophisticated linguistic comprehension and generation faculties [148], these models exhibit notable limitations regarding real-time information acquisition, specialized computational operations, and external system interactions. Function calling integration enables these models to access designated interfaces and predefined procedural elements, thereby facilitating current data retrieval, complex operational execution, and fluid integration with complementary technological frameworks [95]. This technological advancement substantially expands the operational parameters of these systems while considerably improving both utilization experience and practical implementation value [171].

3.2 Pre-Call Stage

Preliminary analysis of user inquiries and identification of applicable functional components must be systematically performed to guarantee optimal procedural selection while simultaneously maximizing response precision and operational efficiency throughout the interaction process.

3.2.1 Query Processing. The preliminary phase within the standard function calling operational sequence involves the model's systematic preprocessing of user-submitted inquiries. The core tasks of this stage include understanding the user's request and decomposing the task. Through this process, the model can identify key information and action steps that require further processing, preparing for the subsequent function retrieval stage. A significant challenge in this stage is:

Challenge 1.1: Intent Recognition. The initial challenge lies in recognizing user intent to guide function calling. (Solutions are discussed in Section 5.1)

Understanding user intent is crucial for addressing this challenge. Interpreting user requests to discern their intentions necessitates learning a mapping from the instruction space to the model's cognition space [137, 189]. These approaches employ external user-specific modules or prompts to incorporate users' preferences, styles, and personal information into the generated content. For example, GeckOpt [23] refines tool selection by incorporating intent-driven gating. Additionally, some research [108, 166] advocates for leveraging user feedback to dynamically adapt the model to individual users, presenting a promising avenue for personalizing LLMs. To the best of our knowledge, recognizing user intent in most function-calling applications predominantly relies on the inherent capabilities of large models and the strategic construction of prompts.

3.2.2 Function Retrieval Processing. After query processing, it is crucial to select relevant function candidates for subsequent steps. Given that a large commercial enterprise may have thousands of functions, failing to perform an initial screening could result in an overwhelming number of choices. This introduces a new challenge:

Challenge 1.2: Function Redundancy. When multiple functions serve similar purposes, the system faces decreased efficiency and slower response times due to redundant function choices that complicate the selection process. (Solutions are discussed in Section 5.1)

Initial function retrieval typically leverages the features of functions to address this challenge. It is important to distinguish between function selection and initial function retrieval: function selection involves choosing the correct function from a set of candidates, whereas retrieval pertains to identifying potential candidates. Some papers conflate these concepts. In this article, 'initial function retrieval' denotes explicitly the process of identifying potential candidates. A practical retrieval module is crucial for selecting the top-K suitable function candidates from a large pool in scenarios with numerous functions. The introduction of this module bridges the gap between the capabilities of LLMs and practical input size limitations, as not all functions can be directly input into LLMs. Traditional term-based methods, such as BM25 [116], represent documents and queries as high-dimensional sparse vectors. For instance, Gorilla [102] combines BM25 and GPT-Index to construct a tool retriever for tool retrieval. Semantic similarity can be calculated using language model embeddings and cosine similarity. For instance, Confucius [25] trained a SentenceBERT model as a tool retriever, enhancing the efficiency of relevant tool retrieval. CRAFT [183] guides LLMs to generate fictitious tool descriptions based on a given query, utilizing these semantic details for searching. Similar to traditional information retrieval domains, some topological information can also be incorporated. For example, COLT [110] proposes a novel tool retrieval approach using **Graph Neural Networks (GNNs)** to ensure the completeness of the retrieved functions.

3.3 On-Call Stage

After preparing the tasks and selecting a set of function candidates, the next typical step is to proceed with the function calling operation. This involves addressing two sub-problems: 'When to call' and 'How to call.' In practical implementations, the distinction between these two parts is often quite blurred, typically manifesting as a single-step generation by the LLM. However, to facilitate a comprehensive understanding of the entire function calling process, we discuss these two steps separately. Nevertheless, in practical applications, LLMs usually complete these steps **in a single operation** [95].

3.3.1 When to Call. After function retrieval, the model needs to assess whether a function call is required to complete the user's task. This decision depends on the model's deep understanding of the task requirements and the context of the dialogue. Correct triggering is crucial to avoid unnecessary function calls and to ensure that necessary function calls occur timely. This presents two challenges:

Challenge 2.1: Missing Calls. The LLM fails to initiate function calls when such actions are required. This lack of proper triggering can lead to missed opportunities for processing or accurately responding to user requests.(Solutions are discussed in Sections 5.1 and 5.2)

Challenge 2.2: Unnecessary Calls. The LLM initiates unnecessary function calls when not required by the user's current task. As shown in ChemAgent's [182] evaluation, unnecessary function callings may underperform base LLMs on general tasks if used indiscriminately. This not only results in inefficiencies and a waste of resources but may also mislead the model's output. (Solutions are discussed in Sections 5.1 and 5.2)

3.3.2 How to Call. Once a function call is triggered, the LLM needs to decide which function to call and extract the necessary parameters from the user's query. This process generally includes identifying keywords and phrases in the user's language and converting them into the parameters required for the correctly chosen function. This step is critical to ensuring the accuracy of the function call. However, the selection and extraction of parameters can encounter the following challenges:

Challenge 3.1: Missing/Illegal Parameters. Missing/Illegal parameters issue arises when the parameters extracted from the user's input are inadequate or inappropriate for executing the intended function. This can lead to potential failures in function execution or the necessity for the model to solicit additional input from the user, thereby complicating the interaction and possibly affecting user satisfaction. (Solutions are discussed in Sections 5.1, 5.4, 5.5, and 5.6)

Challenge 3.2: Function hallucination. The phenomenon of functional fabrication manifests when language models erroneously invoke non-applicable or entirely fictitious operational component, or populate parameters lacking legitimate existence. Such aberrations potentially generate inaccurate system responses and compromise procedural reliability, consequently inducing user dissatisfaction and diminishing confidence in the technological framework.(Solutions are discussed in Sections 5.1, 5.2, and 5.3)

Furthermore, linguistic reference elements within user communications–including temporal indicators such as "today" and pronominal forms like "he" or "it"–necessitating contextual decoding for parameter determination, constitute an additional computational complexity.

Challenge 3.3: Pronouns Resolving. Accurate disambiguation of referential elements within user-initiated linguistic constructs represents a substantial technical impediment, attributable to the inherently fluid and context-contingent characteristics of natural language. Inadequate resolution of such referential ambiguities frequently precipitates erroneous parameter extraction mechanisms, thereby fundamentally compromising the operational integrity of function invocation protocols.(Solutions are discussed in Sections 5.4 and 5.6)

Moreover, consequent to their developmental conditioning within unconstrained textual corpora and intrinsically sophisticated architectural configurations, large language models manifest inherent computational constraints that significantly impact performance metrics, particularly regarding temporal response parameters and precision indicators. This operational limitation attains particular significance when these computational frameworks are deployed within specialized implementation domains necessitating exactitude and expeditious output generation.

Challenge 3.4: LLM Inherent Limitations on Latency and Accuracy. Large language models trained on broad, non-domain-specific corpora and implemented with computationally intensive architectures face intrinsic limits in specialized settings. Even without external tools, they may exhibit increased and unpredictable response latency, difficulty disambiguating domain-specific terminology, and omission of key constraints during reasoning. These issues stem from model capacity, training distribution mismatch, and inference cost, rather than from multi-function workflows. (Solutions are discussed in Sections 5.3, 5.5, and 5.7)

Furthermore, when confronted with user-initiated operational requisitions, the comprehensive fulfillment of such directives may necessitate sequential multiple-function activation protocols, exemplified by conference facility reservation processes wherein the computational framework must initially execute database interrogation procedures regarding facility availability parameters before subsequently implementing selection algorithms conforming to specified conditional criteria. This operational complexity introduces the following methodological challenge.

Challenge 3.5: Tool-Orchestrated Multi-Call Workflow. The execution framework may need to maintain end-to-end task understanding across multiple function calls, whether in parallel or in sequence. This requires stable intent tracking, accurate parameter passing, consistent state management, and alignment of intermediate results, while performing task decomposition and scheduling that minimize the critical path and manage external I/O latency.(Solutions are discussed in Sections 5.3, 5.5, and 5.7)

Furthermore, consequent to the inherently iterative conversational structures characteristic of human–machine interactions and the potentially extensive epistemological repositories maintained by individual users, a significant methodological impediment emerges regarding informational continuity management.

Challenge 3.6: Effective Context Management. Comprehensive administration of this expansive informational continuity framework constitutes an operational imperative of paramount significance, functioning to mitigate potentialities of critical data attenuation while simultaneously ensuring the maintenance of response homogeneity throughout the duration of interactive procedural engagements. Inadequacies in this area can significantly impair the precision and user experience of function calling. Previous context may include essential user information, domain knowledge, or the history of multiple rounds of dialogue, including the results of functions executed in previous turns.(Solutions are discussed in Sections 5.2 and 5.6)

After parameter extraction, the model integrates these parameters into the chosen function. During this phase, the model may need to use built-in logic or interact with the user to address any parameter deficiencies. Once this step is completed, the model is prepared to send the function and its parameters, thus concluding the function calling process.

3.4 Post-Call Stage

The objective of the post-call stage in the function-calling process is to execute the function and generate an appropriate natural language response. This phase involves converting the natural language outputs of the LLM into executable real-world functions. Once converted, these functions are sent for execution. Subsequently, the process also requires analyzing the execution results, managing instances of execution failures, and reconciling any discrepancies between the function's results and the user's original query and expectations. The challenges faced in this stage include:

Challenge 4.1: Execution Result Mismatch. Even when functions are correctly called, the outcomes may not align with user expectations, leading to mismatches in execution results. These mismatches can occur due to constraints inherent in the function's logic, malicious attacks [163, 186] on the function, or inconsistencies in the quality of function description.(Solutions are discussed in Section 5.5)

Challenge 4.2: Irrelevant Information Overload. Some functions return some helpful, relevant information and an excessive amount of irrelevant information due to their design for comprehensiveness, which can lead to verbose responses and significant redundancy. This issue necessitates effective strategies for pruning irrelevant details to enhance the clarity and utility of the output.(Solutions are discussed in Section 5.5)

Challenge 4.3: Mismatch Between Real-World Functions and LLM-Generated Results. Functions filled by an LLM generally cannot be directly executed due to a significant mismatch

between the semantic space and the code space, making mapping a considerable challenge. For instance, to query the weather in "Hangzhou" on "January 1st, 2024", the function must be formatted correctly for the server. It may require a syntax like "GetWeather(date="2024.1.1", city="007")" instead of "GetWeather(date="January 1st, 2024", city="Hangzhou")" generated by LLMs.(Solutions are discussed in Sections 5.4 and 5.5)

Challenge 4.4: Execution Failure. Notwithstanding precise functional invocation and adequate parameterization protocols, operational execution failures may nevertheless manifest due to multifarious technical impediments. Execution anomalies potentially derive from server-side computational aberrations, structural incongruities between anticipated data architecture specifications and actual information configurational manifestations, or inherent constraints within functional design paradigms that preclude efficacious processing of boundary conditions or unanticipated input typologies. Such operational vulnerabilities necessitate the implementation of sophisticated exception management infrastructures and alternative procedural pathways to preserve systemic operational integrity and maintain user confidence metrics.(Solutions are discussed in Section 5.5)

The comprehensive resolution of aforementioned methodological impediments would facilitate enhanced linguistic model performance vis-à-vis complex user directive processing, precise functional invocation execution, and appropriate response generation while simultaneously preserving contextual relevance parameters and output consistency indicators. Through transcendence of current operational limitations, computational frameworks would attain superior capability regarding nuanced query interpretation, intricate workflow administration, and reliable output production across diverse interaction modalities. This augmented functional capacity would engender robust operational support spanning multitudinous application scenarios, encompassing rudimentary task automation implementations through sophisticated multi-iterative conversational exchanges, ultimately enhancing user satisfaction indices and systemic reliability coefficients. The resultant performance optimization would concurrently facilitate seamless integration capabilities with extant technological infrastructures while accommodating emergent utilization paradigms. Subsequent analytical sections shall elucidate detailed methodological approaches addressing these operational challenges through the prismatic perspectives of exemplar construction methodologies, parameter optimization protocols, and deployment architectures, examining specific procedural frameworks and implementation heuristics for each operational dimension.

4 Enhancing LLMs to have the Ability: Sample Construction and Fine-Tuning

In the previous section, we discussed the necessary steps for function calling. This chapter addresses how pre-trained LLMs under natural language settings can be endowed with function calling capabilities, as illustrated in Figure 2. As shown in Table 2, we present a systematic review of sample construction methods and fine-tuning strategies, ranging from function collection approaches to critical considerations in the training process.

4.1 Function Collection

The initial step involves collecting functions, including the function object–abstract entities consisting of the function name, parameters, and execution logic–and their descriptions. Real-world functions are often disorganised, and descriptions can be chaotic. Aside from manual construction, function objects and their descriptions can also be generated using large-scale LLMs like GPT-4 [1],LlaMA 70B [145] or Qwen [5, 144]. However, our experience suggests that such generated functions often lack diversity. Alternatively, data mining techniques can extract diverse function objects from the web, with descriptions supplemented by powerful LLMs if missing. By assembling a sufficient number of functions, the model can effectively grasp the timing of function triggers, thereby addressing **Challenge 2.1**.

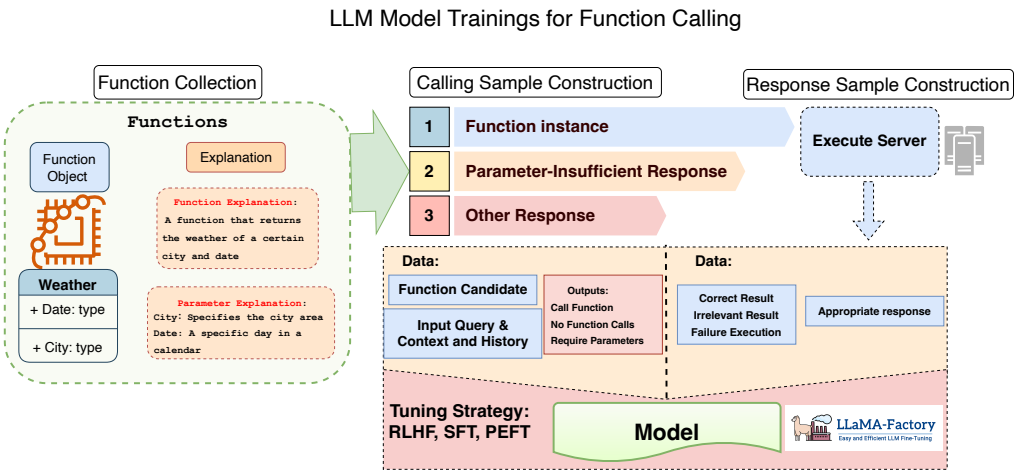

Fig. 2. Exemplar methodological framework for augmentation of linguistic model functional invocation capacities: This schematic representation delineates one potential procedural implementation trajectory through exemplar construction and parametric optimization methodologies, comprising three principal operational phases: (1) Functional Entity Aggregation, wherein computational procedural constructs and their corresponding ontological descriptors are systematically accumulated and categorized; (2) Invocation Sample Architectural Development, potentially encompassing functional execution instantiations, parameter-deficient scenarios, and heterogeneous response configurational paradigms; and (3) Output Sample Structural Formulation, illustrating one methodological approach toward mapping execution resultant states to appropriate linguistic response generations. While other approaches exist (e.g., LlamaFactory [198]'s unified training framework), this pipeline demonstrates common components that practitioners might consider when implementing function calling capabilities.

Table 2. Sample Construction and Fine-Tuning Strategies for Function Calling in LLMs

Sample Construction and Fine-Tuning	Function Collection (Section 4.1)	**Manual Construction:** Human-crafted functions **LLM Generation:** Usage of LLMs for automated function creation **Web Mining:** Diverse function extraction from web
	Sample Construction (Section 4.2)	**Text Representation:** Toolformer [122], ToolGen [153] **Token Representation:** Toolformer [122], ToolGen [153] **Multi-turn Interaction:** GraphQL-RestBench [119], Hammer [69]
	Fine-tuning Strategies (Section 4.3)	**SFT:** ToolGen [153], RAIT [120], Nye et al. [93], Andor et al. [3], Liu et al. [70], Allamanis et al. [2], Barone et al. [83], Liu et al. [75], Liu et al. [73] **PEFT:** GPT4Tools [176], CITI [36], Toolformer [122], Li et al. [58], Wei et al. [162] **RL & RLHF:** WebGPT [89], TaskMatrix.AI [65], MADAC [63], GopherCite [112], Kojima et al. [52], DPO [113], Christiano et al. [14], Manduzio et al. [80]
	Critical Emphasis (Section 4.4)	**Data Quality:** Focus on data diversity and verification rather than volume **Model Scaling:** Larger models show better function calling capabilities **Capability Balance:** Need to maintain general abilities while enhancing function calling

4.2 Function Calling Sample Construction

We can construct sample mapping queries to outputs with a sufficient collection of functions. Real-world function calls involve returning the corresponding function and parameters, awaiting execution by the system and server. In cases where user-provided parameters are insufficient, the model may generate queries asking for additional information. Samples must, therefore, cater to these scenarios by including a variety of input-output pairs. Only when provided with a sufficient number of samples containing inadequate parameters, can the model learn when to request missing parameters from users, thereby partially addressing **Challenge 2.2**. Functions can be represented either as text or tokens in the model. Text representation provides more flexibility

and semantic information but requires more token space, while token representation (as adopted by Toolformer [122] and ToolGen [153]) is more compact and computationally efficient but may have limited expressiveness. These two approaches can be combined - using token representation during training for efficiency while converting to text format during inference for better interpretability and interaction. For instance, both Toolformer and ToolGen encode functions as special tokens during training, but can present them in natural language format during interaction with users.**Challenge 3.1**.

Inputs typically consist of function candidates and a natural text query generated by robust LLMs based on original functions and descriptions. Outputs are crafted to match the varying requirements of real-life function-calling scenarios. For instance, in situations where function calls are not necessary, it is essential to construct examples to guide the model in understanding when to rely on its capabilities to respond. This approach could address **Challenge 2.2**. Moreover, samples can be constructed to simulate multi-turn interactions to enhance multi-turn function calling capabilities [12]. GraphQL-RestBench [119] further advances sequential function calling by introducing structured API schemas and response mapping, focusing on real-world REST API scenarios where functions have complex interdependencies.

To further improve robustness in function selection and address **Challenge 2.2**, recent work like Hammer [69] introduces specialized techniques such as function masking and dataset augmentation. This approach specifically targets naming convention sensitivity issues that often mislead models, while demonstrating state-of-the-art performance even in resource-constrained on-device deployment scenarios.

4.3 Fine-Tuning Strategies

LLMs trained in natural language can perform function calls by designing prompts to enable this capability, as discussed in other sections. However, this approach is often insufficient because the model's reasoning and acting abilities depend entirely on the provided prompts, and the model itself does not self-optimize or adapt to new environments. Therefore, additional tuning is essential, mainly when new behaviors are challenging to learn and require more than just a few examples. Before fine-tuning LLMs, constructed function calling samples and response samples can undergo a uniform format cleaning process according to the specific usage, which typically includes a series of cleaning strategies, such as data deduplication [2], syntax normalization [83], format standardization [75], and error correction [73]. Subsequently, some LLM fine-tuning strategies such as PEFT (parameter-efficient fine-tuning), and RLHF (reinforcement learning with human feedback) can be employed to optimize model performance via constructed and cleaned samples.

Specifically, parameter θ^* is learned by maximizing the likelihood of the output samples corresponding to the dataset constructed, as discussed in previous sections. The objective function calculates the expected value of the product of action probabilities for all queries q_i and their corresponding outputs a_i^* within the dataset $\mathcal{D}$, given the auxiliary cues $x_{i,t}$ and the conditions of the queries. Specifically, the objective function is defined as $\theta^* = \arg\max_\theta \mathbb{E}_{(q_i,a_i^*)\in\mathcal{D}}[p_\theta(a_i^* \mid x_i, q_i)]$, where $p_\theta(a_i^* \mid x_i, q_i)$ represents the probability of output given the query and auxiliary cues. Beyond direct imitation learning, additional information and tasks can be integrated to enhance the training process. For example, Nye et al. [93] use execution traces as a form of supervision to improve reasoning abilities. In contrast, Andor et al. [3] apply heuristics to gather supervised data, facilitating the fine-tuning of language models. RAIT [120] combines instruction-tuning of code small language models with retrieval-augmented tool usage, enabling systematic problem-solving in process engineering. ToolGen [153] injects tool information through fine-tuning LLMs with tool descriptions as inputs and their corresponding tokens as outputs, directly incorporating tool knowledge into model parameters. Additionally, the integration of LoRA can further refine these

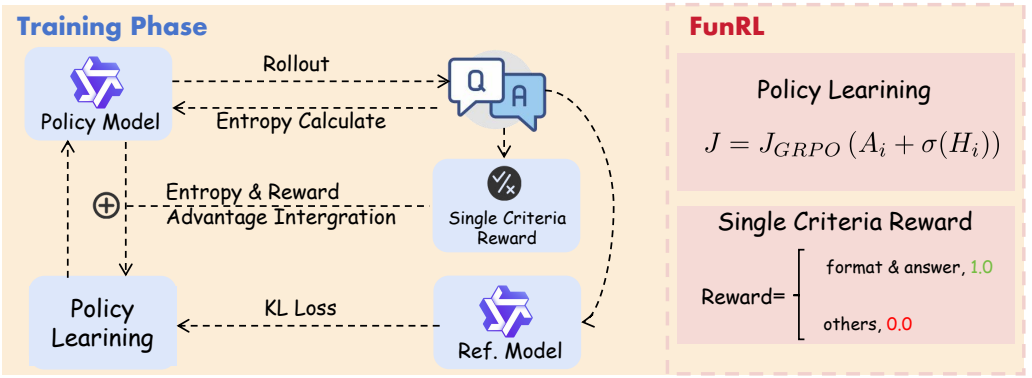

Fig. 3. Exemplar reinforcement framework for language model function calling from FunRL [33]. This schematic diagram illustrates a potential implementation trajectory for a function calling reinforcement method through reward design and advantage estimation.

approaches. For example, GPT4Tools [176] advance the capabilities of open-source LLMs by incorporating LoRA optimization techniques into fine-tuning, utilizing datasets composed of tool usage instructions produced by ChatGPT. CITI [36] proposes a novel Mixture-of-LoRA adapter to important and selectively fine-tunes unimportant components, enabling LLMs to enhance their tool-utilizing capabilities while preserving general performance across multiple tasks.

Reinforcement Learning (RL) in function calling enables LLMs to interact with external tools or APIs within a task directly, enhancing their ability to perform specific functions based on given inputs. For instance, Toolformer [122] utilizes RL-based techniques to bootstrap examples of tool use, improving the language model's effectiveness in function calling. Incorporate reward-driven RL frameworks, Zhang et al. [190] propose Nemotron-Research-Tool-N1, which combines reinforced reasoning with tool selection through reward-guided mechanisms in multi-tool environments. Similarly, ToolRL [104] demonstrates that reward signals alone can effectively guide tool learning and composition. **Optimal Tool Calls (OTC)** [151] further optimizes both answer correctness and tool calling efficiency through RL, reducing unnecessary invocations while maintaining accuracy. ReTool [22] integrates code interpreter execution within the reasoning process, using task outcomes as rewards to guide strategic tool use. Additionally, holistic agentic frameworks like VerlTool [45] propose unified architectures for multi-turn tool use with asynchronous rollout capabilities. In industry application, FunRL Framework [34], as shown in Figure 3 comprises Entropy Calculation for helping the model explore thought patterns conducive to function calling, and Reward Design for facilitating more efficient training for possessing foundational function-calling capabilities.

Reinforcement Learning from Human Feedback (RLHF) [1, 14] further enhances this approach by aligning the language model more closely with complex human preferences and values that are challenging to capture with hardcoded reward functions. This proves particularly useful in function-calling scenarios [58] where the model interacts with external systems or tools to perform tasks such as computation, information retrieval, or navigating web interfaces. For example, WebGPT [89] employs a text-based browser to answer questions using internet searches, optimizing browsing actions and answer quality through RLHF. Similarly, GopherCite [112] is fine-tuned with RLHF to cite supporting evidence when answering questions and to abstain when unsure. TaskMatrix.AI [65] leverages RLHF to incorporate human feedback dynamically, enhancing the model's ability to navigate and coordinate the use of multiple tools and APIs. Recent work [80] proposes an efficient framework for training smaller models in function calling capabilities,

using larger models to generate training data through step-by-step reasoning chains [52, 162] and **Direct Preference Optimization** (**DPO**) [113]. The safe multi-agent RL framework proposed by MADAC [63], which ensures state-wise safety constraints while achieving optimal performance, could potentially provide insights for developing safe function calling systems. This approach demonstrates that focused training on specific reasoning tasks can achieve strong performance even with reduced model sizes, addressing deployment constraints.

In addition to their direct application [70], RL methods are crucial for repairing bad cases in real-world function-calling scenarios within industrial settings. There is a perceptual discrepancy between LLMs and human interpretation of queries. For example, when users search for 'flights from Los Angeles to Hangzhou on the evening of August 30th,' they might also consider flights in the early hours of August 31st as 'evening' flights. However, if an LLM strictly interprets the query, it may miss these flights. Relying solely on explicit data to resolve these discrepancies is impractical, as training LLMs to encompass all potential user intents is unfeasible. Furthermore, it is difficult to capture complex intentions through simple inputs due to the limitations in prompt length. Therefore, adjusting based on human feedback becomes crucial. RLHF is essential for function-calling learning because it allows for direct learning from human feedback, correcting errors, and refining system responses. This technology is critical for addressing habitual mistakes, misentered parameters, and even hallucinations in user inputs. Fine-tuning strategies effectively address several key challenges. For **Challenge 3.1** and **Challenge 3.2**, approaches like ToolGen [153] demonstrate that directly incorporating tool knowledge into model parameters through fine-tuning can significantly reduce parameter errors and hallucination. To address **Challenge 2.2**, methods like GPT4Tools [176] and CITI [36] show that LoRA-based selective fine-tuning can effectively reduce unnecessary function invocations while maintaining model performance. Additionally, for **Challenge 4.1** and **Challenge 4.4**, RLHF approaches demonstrated by WebGPT [89] and MADAC [63] provide effective frameworks for handling execution failures and improving result accuracy through human feedback.

4.4 Critical Emphasis

Based on practical implementations, we emphasize that data quality (and variety) plays a more crucial role than data quantity in both data construction and fine-tuning phases, given the intricate nature of function calling tasks. To validate this hypothesis, we conducted a simple experiment: we selected a subset of 1,000 samples from the BFCL [101, 102] training dataset for supervised fine-tuning using LLaMA Factory [198], and used BFCL's **Abstract Syntax Tree** (**AST**) test category for evaluation. We chose AST tests as they enable offline evaluation of function calling capabilities without requiring external API access. We applied LoRA fine-tuning with a rank of 8 and a learning rate of 3e-4, training for 3 epochs. We specifically chose Qwen1.5-7B-Instruct [5] as our base model to ensure experimental integrity, as this older version predates the release of the Gorilla dataset, thus eliminating potential data contamination concerns. To investigate the minimal data requirements for achieving satisfactory function calling capabilities, we conducted experiments with varying training sample sizes.

As shown in Figure 4, our experimental results demonstrate that model performance quickly reaches a plateau after processing approximately 400 training samples. Moreover, to evaluate the impact of data quality, we further incorporated the Pandora dataset [103], which is a high-quality subset filtered from the Glaive v2 dataset [29]. As shown in Figure 5, under the same data scale (400 samples), models fine-tuned on pandora dataset consistently outperform those trained on the base data. These phenomenons suggests two important insights: First, the function calling ability can be effectively learned with a relatively small amount of high-quality data, indicating that the model can quickly grasp the underlying patterns of API usage and parameter formatting; Second, simply increasing the training data volume beyond this point yields diminishing returns, which

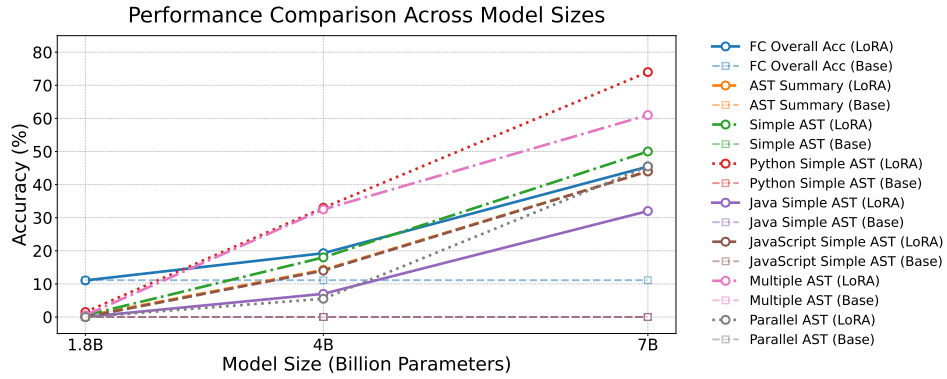

Fig. 4. Performance trends across different model sizes. Base models show near-zero function calling capabilities across all scales, indicating this ability is unlikely to emerge naturally during pre-training. After LoRA fine-tuning, performance exhibits clear scaling law characteristics, with particularly significant improvements from 4B to 7B.

Fig. 5. Performance comparison between the base data and the pandora data. The pandora data, which provides higher-quality and more diverse samples, yields consistently improved accuracy across multiple function-calling metrics.

emphasizes the importance of data quality over quantity in function calling tasks. This finding has significant practical implications for efficient model training and resource utilization in real-world applications.

As shown in Figure 6, we conducted another experiment to investigate the impact of model size on function calling capabilities. We selected three Qwen models of different sizes (1.8B, 4B, and 7B) and evaluated their performance on BFCL's AST tests, both with and without fine-tuning. Our experiments reveal two key findings: First, base models without fine-tuning demonstrate minimal function calling capabilities, suggesting this skill rarely emerges naturally during pre-training. Secondarily, the performance trajectories subsequent to Low-Rank Adaptation parametric optimization methodologies manifest characteristic computational scaling law phenomenology, with particularly

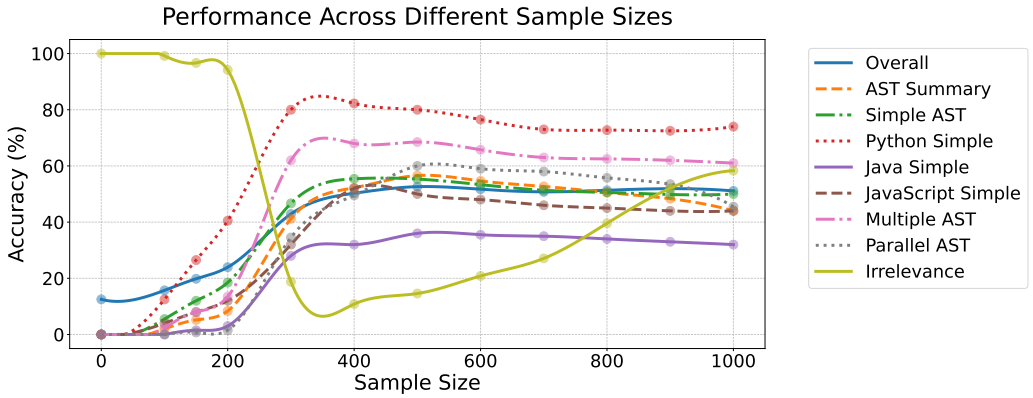

Fig. 6. Performance comparison of different models across various sample sizes. The results show that Python Simple achieves the best performance with a peak accuracy of 82.25% at 400 samples, while Multiple AST and AST Summary also demonstrate strong performance above 60% accuracy.

significant efficacy amplifications observed at the 7B parameter quantification threshold, indicative of enhanced functional invocation capability acquisition correlating positively with architectural volumetric expansions. This scaling pattern topology reveals critical epistemological insights regarding model dimensionality requirements for functional invocation operational tasks. Whereas computational frameworks exceeding 7B parametric quantities demonstrate promising performance enhancement coefficients, reduced-scale architectures (sub-7B configurations) exhibit persistent inadequacy in attaining comparable operational efficacy despite substantial training corpus augmentation. This empirical observation suggests that conventional optimization methodologies may prove insufficient for reduced-scale models to acquire robust functional invocation capabilities. For more resource-efficient deployment within computationally-constrained operational environments, subsequent investigative endeavors should explore enhanced methodological approaches for reduced-scale architectures, including externalized memory mechanisms to compensate for inherent capacity limitations, specialized training paradigms such as incremental complexity progression methodologies, or architectural reconfiguration strategies specifically engineered for functional invocation operational contexts. This empirical finding underscores the necessity of optimizing equilibrium between computational efficiency metrics and functional invocation capability coefficients, particularly within deployment scenarios where expanded-scale architectural implementations may present practical implementation constraints.

In practice, intensive function call training tends to compromise the model's general capabilities, particularly in text generation. As demonstrated in our experiments, this tradeoff between specialized function call abilities and general language capabilities presents a significant challenge. To investigate this phenomenon more systematically, we conducted experiments with Qwen-7B under three scenarios: the base model, function calling **fine-tuned** (**FC**), and mixed training with both function calling and natural language samples (Mixed). As illustrated in Figure 7, our evaluation using AST overall accuracy, CNN/Daily Mail dataset [124]'s RougeL score, and BoolQ accuracy [15] reveals an interesting pattern: while function calling fine-tuning substantially improves the AST performance, it appears to come at a cost to certain general language capabilities, particularly in question-answering tasks. The text generation ability, however, remains relatively stable across different training configurations, suggesting a complex relationship between different language capabilities. To address this challenge, we suggest incorporating domain-outlier data samples and

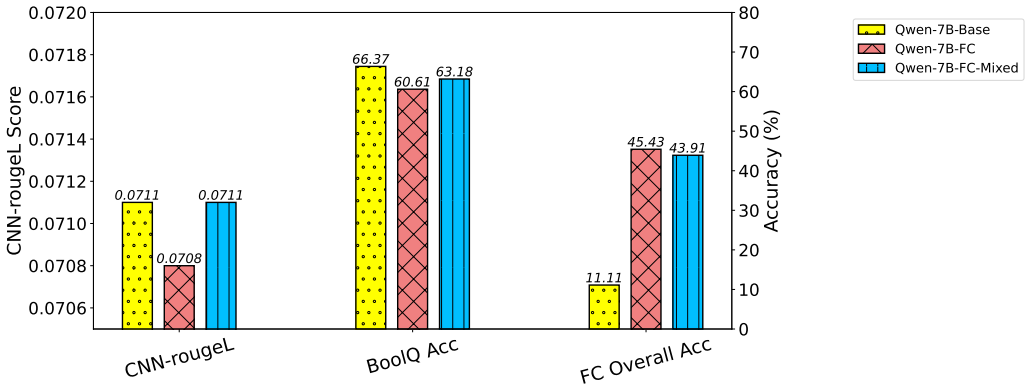

Fig. 7. The potential seesaw effect across different language capabilities. After fine-tuning Qwen-7B with function calling data, we observe a substantial improvement in function calling accuracy (FC Overall Acc increases from 11.11% to 45.43%). Meanwhile, the model shows varying performance changes in other capabilities: a decrease in question-answering ability (BoolQ accuracy drops from 66.37% to 60.61%) and relatively stable performance in news generation (CNN-rougeL maintains around 0.071). This pattern might suggest a seesaw effect between specialized function calling capability and general language abilities, particularly in question-answering tasks, though more systematic studies would be needed to confirm this hypothesis.

natural language text [97] alongside function call examples during fine-tuning [142]. This mixed training approach attempts to maintain model versatility while developing function call competence, aligning with successful strategies that blend different types of training data [114]. While our initial results show promise in balancing these capabilities, further systematic studies would be needed to fully understand and optimize this relationship. Furthermore, concepts from lifelong learning and curriculum learning [156, 193] may offer promising directions for achieving a better balance between specialized and general capabilities, particularly crucial for practical applications where maintaining both sets of abilities is essential.

Some studies recommend sample augmentation [136] to improve tuning outcomes. APIGen [75] presents an automated pipeline for generating high-quality, verifiable function-calling datasets through a three-stage verification process, demonstrating that models trained on such carefully curated data can achieve superior performance even with relatively small parameter counts compared with larger models like GPT-4. ToolACE [73] presents an innovative self-evolving data synthesis pipeline that leverages multi-agent interactions and dual-layer verification to generate high-quality function-calling training data, enabling 8B parameter models to achieve GPT-4-level performance on standardized benchmarks. These approaches and the strategies discussed form a comprehensive methodology for tuning LLMs to handle sophisticated function-calling tasks effectively.

What's more, the ultimate goal for large models is to generate appropriate natural language text responses based on user queries. Data for mapping function return results to suitable natural language responses is therefore necessary.

5 Implementing LLMs for Function Calling: Deployment and Inference

After constructing the sample and fine-tuning the large model, this section examines common deployment approaches that implement the three-stage pipeline (pre-call, on-call, post-call) outlined in Section 3. The deployment process for function-calling LLMs typically involves multiple inference steps aligned with these stages. While some implementations include an initial inference step for

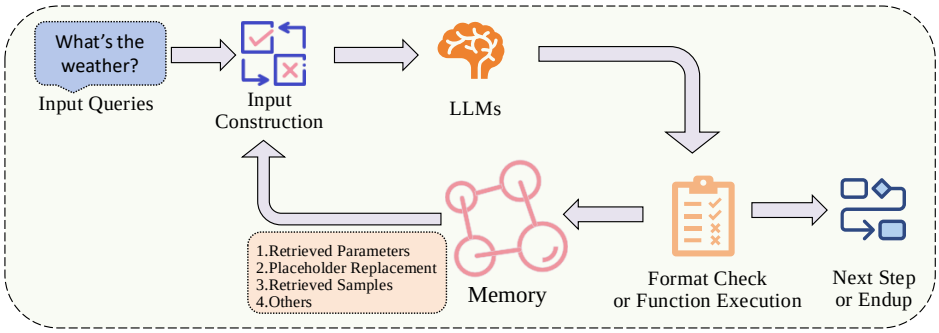

Fig. 8. A typical deployment of LLM for function calling stages: The Flow through Input Construction, Memory Integration, and Output Format Validation (Function Execution). Note that actual implementations may vary in practice.

Table 3. Deployment and Inference Strategies for Function Calling in LLMs

Deployment and Inference	Task Planning (Section 5.1)	**Foundational Planning Mechanisms:** ReAct [179], ToolFormer [122], Reverse Chain [192], AVATAR [165], DEPS [159], LLM-MCTS [196], MACT [200], TACO [79], PAE [201], SCIAGENT [164], Agent Laboratory [123] **GUI-based Approaches:** AppAgent [187], OS-ATLAS [168], AndroidLab [173], Ponder [158], OS-Genesis [141] **System Optimizations:** Orca [84, 87], Memgpt [98], AIOS-Agent [28], SpecInfer [82], PEOA [135], LLM-Tool Compiler [132] **Error:** LLM-Planner [133], ToolChain* [202], TPTU [118], Buckets [13], AMOR [30] **Tree-based:** ControlLLM [76], PLUTO [38], Toolink [106], TPTU-v2 [53], α-UMi [127] **Adaptive:** COA [24], DEER [31], SOAY [157], ProgPrompt [130], AutoTOD [191], MATMCD [126], CC-PP [37], AVT [175], K-agents [10], Agent-Pro [191], Inner [74]
	Context Engineering (Section 5.2)	**Text Integration:** FunReason [34] Example demonstrations [136], Four-shot prompting, Function definitions, Docstrings **Query-based Retrieval:** Ask-when-Needed [155], Interactive refinement **Multimodel Context Management:** MLLM-Tool [150], VisionGPT [49]
	Function Generation (Section 5.3)	Grammar control [100, 121], Knowledge guidance [9] Attribution [203], Feedback [171], Dialogue refinement [46] Multi-agent coordination [32], Task proposal [201], Experience transfer [105]
	Function Mapping (Section 5.4)	**Resolution:** Rule-based [55, 72], Knowledge reasoning [188], LLM mapping [56] **Alignment:** Dictionary mapping [138], Semantic matching [68], normalization **Validation:** Gatekeeper [146], PrivacyChecker [154]
	Response Generation (Section 5.5)	**Initial Generation:** Placeholder results [35, 99, 122], Function unpredictability [179] **Templates:** Structure format [102, 109], Formatting [57], Signatures [136] **Review:** Validation [109, 134], Agent correction [43, 129], Feedback [39, 90, 170] **RAG:** Example retrieval [26], System mapping [11, 66, 91, 115, 117, 136, 184]
	Memory Scheme (Section 5.6)	**Memory Structure:** Hierarchical structure and storage [199], Task-related symbolic memory [178], Three-layered memory architecture [61], Persistent memory stream [64] **Memory Management:** Self-controlled memory mechanism [64], Memory control system [64, 86], Multi-agent experience storage [62] **Memory Retrieval:** Cross-conversation memory retrieval [199], LSH-based indexing mechanism [86], Similarity-based retrieval [197], Efficient memory access [71] **Memory Processing:** Thought-based memory storage [71], Trajectory-as-exemplar framework [197], State abstraction mechanism [197], Knowledge triplet [86]

query understanding and decomposition using an intent model, this preprocessing phase continues to evolve in industrial applications and represents an area for future exploration. The core process of handling user queries and functions directly follows a well-defined workflow. As shown in Figure 8, a typical deployment involves input construction with the query and contextual information (pre-call), LLM-based function generation (on-call), and format check or function execution (post-call). The Memory component maintains essential context across these stages. This represents one common approach among various possible implementation patterns that practitioners have explored. As detailed in Table 3, we provide a comprehensive overview of deployment and inference strategies, covering aspects from task planning to memory schemes.

5.1 Task Planning and Compiler

5.1.1 Foundational Planning Mechanisms. Understanding user intent is insufficient for complex function calling, so task planning is required, progressing from chain-of-thought prompting and fine-tuning to structured multi-agent and search-based frameworks. Early methods established decomposition and tool use with ReAct [179] and ToolFormer [122], while Reverse Chain [192] added target-driven backward reasoning for controlled multi-API planning without fine-tuning. Recent systems adopt multi-component architectures and iterative refinement: Agent Laboratory [123] coordinates specialized agents (PhD, Postdoc, and Professor) using function calling, reflective dialogue, and stage-wise human feedback for scientific workflows, paralleling AVATAR's [165] actor–comparator design; DEPS [159] provides description-based interactive planning; LLM-MCTS [196] integrates Monte Carlo Tree Search with memory, using LLMs as world model and heuristic. These planners transfer to diverse applications: MACT [200] uses iterative planning between planning and coding agents for complex table analysis; TACO [79] supplies synthetic Chain-of-Thought-and-Action traces for multimodal tasks; PAE [201] boosts zero-shot generalization with context-aware planning and evaluation; SCIAGENT [164] performs tool-augmented scientific reasoning across domains; Ning et al. [92] pair Code Property Graphs with LLM semantics for agent code defect detection; FlowAgent [128] introduces a **Procedure Description Language** (**PDL**) to bridge natural language flexibility with code-like procedural control. Overall, the field evolves from basic CoT (ReAct [179], ToolFormer [122]) to advanced, domain-adapted planners (AVATAR [165], DEPS [159], LLM-MCTS [196], MACT [200], TACO [79], PAE [201], SCIAGENT [164], Ning et al. [92], FlowAgent[128]) that combine structured decomposition, iterative refinement, search, and specialized representations.

5.1.2 GUI-Based Approaches. Recent work has explored GUI-based approaches to enhance function calling capabilities. AppAgent [187] proposes a two-stage training paradigm combining GUI grounding pre-training and action fine-tuning to enable LLMs to understand and interact with mobile applications. OS-ATLAS [168] further extends this by introducing a foundation action model for generalist GUI agents, unifying the action space across different platforms. These approaches demonstrate the potential of combining API and GUI-based methods for more comprehensive task execution capabilities. AndroidLab [173] provides a systematic benchmark framework for evaluating such autonomous agents in real-world mobile environments. Ponder & Press [158] proposes a divide-and-conquer visual GUI agent framework that uses only visual input, featuring a two-stage planning strategy with instruction interpretation and element localization to enable direct and general computer control. OS-Genesis [141] proposes an interaction-driven GUI agent trajectory construction framework that generates training data by exploring environments first, then deriving tasks retrospectively, without human supervision.

5.1.3 System-Level Optimizations. From a system perspective, researchers have explored optimizing LLM execution through compiler and operating system innovations. At the compiler level, Orca [84, 87] proposes a specialized approach to optimize LLM inference through instruction scheduling and memory management, introducing techniques like operation fusion and memory layout optimization to reduce inference latency. Operating system optimizations have also shown promising results. Memgpt [98] presents an operating system design specifically tailored for LLM workloads, providing efficient resource management and scheduling mechanisms. AIOS-Agent ecosystem [28] envisions a revolutionary architecture where LLM serves as the operating system kernel while diverse AI agents function as applications, enabling natural language programming and democratizing software development. These system-level optimizations are crucial for enabling efficient on-device LLM deployment and execution, particularly in resource-constrained environments where memory and computational efficiency are paramount. Recent system optimizations further explore specialized architectures. SpecInfer [82] introduces speculative inference to enhance

serving efficiency through parallel execution. The PEOA [135] introduces a Large **Language Model Operating System** (**LLM OS**) with a modular architecture, where a meta-agent orchestrates an action generator and specialized expert models to break down and solve complex chemical engineering problems, utilizing property graph-based knowledge modelling and teacher-student transfer learning with GPT-4 for improved tool integration and problem-solving capabilities, while managing system resources and tool scheduling in a manner similar to traditional operating systems. LLM-Tool Compiler [132] fuses similar tool operations into unified tasks at runtime, achieving up to 4x improvement in parallel execution while reducing API costs in function calling systems. These works demonstrate the potential of system-level optimizations in improving LLM performance across different deployment scenarios.

5.1.4 Robust Planning through Error Handling. Error handling and recovery mechanisms have been explored to enhance planning reliability. LLM-Planner [133] introduces environmental feedback for plan regeneration during execution failures, while ToolChain* [202] employs decision trees to systematically manage API calls. TPTU [118] and Attention Buckets [13] focus on reducing information loss through structured frameworks and parallel operations. AMOR [30] presents a **finite state machine** (**FSM**)-based framework for function calling that allows autonomous execution and transitions over disentangled modules, enabling process-level human feedback to improve reasoning capabilities through a two-stage fine-tuning approach (warm-up and adaptation).

5.1.5 Tree-Based Decision Making. Tree-structured approaches offer systematic solution exploration. ControlLLM [76] implements Tree of Thoughts with depth-first search on tool graphs. PLUTO [38] uses autoregressive planning with hypothesis trees, while Toolink [106] and TPTU-v2 [53] leverage hierarchical task decomposition. α-UMi [127] extends this through specialized planning-oriented fine-tuning.

5.1.6 Adaptive Planning Strategies. Recent works advance flexible planning and coordination through various approaches. In terms of adaptive planning, COA [24] employs abstract reasoning chains that adapt to domain knowledge, while DEER [31] enhances generalization through dynamic tool sampling. SOAY [157] and ProgPrompt [130] further this adaptability by generating executable code structures tailored to specific execution environments. Building on adaptive planning, several frameworks demonstrate effective task decomposition and multi-agent coordination. MATMCD [126] introduces a multi-agent framework where data augmentation and causal constraint agents collaborate for multi-modal causal discovery. Similarly, AVT [175] decomposes video processing tasks between captioning and arrangement agents, while K-agents [10] coordinates translation and inspection agents for experimental procedures using state machines. Advanced coordination mechanisms are explored in several works. CC-PP [37] employs a two-stage approach combining path heuristics with greedy best-first search for multi-agent coordination under communication constraints. AutoTOD [191] demonstrates effective task planning through instruction-following models, while Agent-Pro [191] introduces dynamic belief management and policy-level reflection. The Inner Thoughts framework [74] enhances multi-party coordination by enabling proactive participation through continuous thought generation and evaluation.

Task planning and system optimization address several key challenges in function calling. For **Challenge 1.1** complex task decomposition and **Challenge 1.2** execution planning, foundational planning mechanisms like chain-of-thought and multi-component architectures provide systematic approaches for task breakdown and execution. For **Challenge 2.1** cross-modal interaction and **Challenge 2.2** system efficiency, GUI-based approaches and system-level optimizations enable effective human-computer interaction and improved computational performance. Additionally, for **Challenge 3.1** error handling and **Challenge 3.2** robustness, various frameworks introduce mechanisms for execution reliability and recovery strategies.

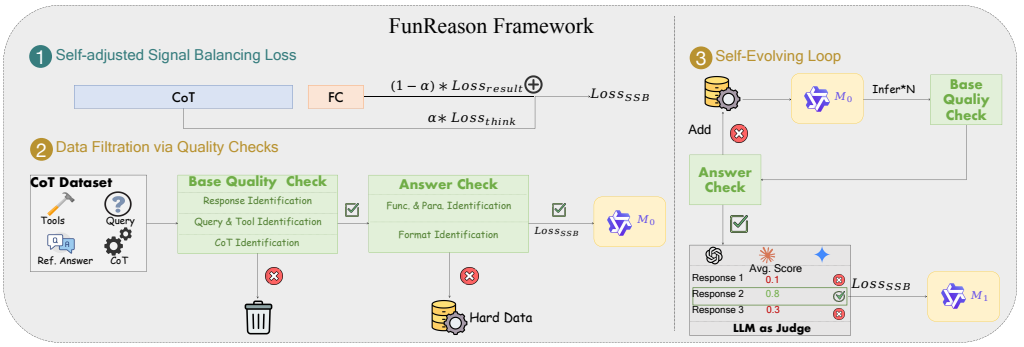

Fig. 9. Exemplar context engineering framework for language model function calling from FunReason framework [34]. This schematic representation, through loss design and a self-evolutionary framework, delineates one potential implementation trajectory for function calling context engineering.

5.2 Context Engineerings

For text-based context engineering, effective prompt construction requires careful integration of few-shot examples, context, and query-based retrieval. NexusRaven [136] demonstrates that retrieving demonstrations from existing query-response pairs, using approximately 16 examples per API function with four-shot prompting, significantly improves function calling success rates. In industry application, FunReason Framework [34], as shown in Figure 9 comprises Dynamic Loss Calculation for helping the model better balance the issue of length disparity between CoT and function call, and a Self-Evolutionary Loop, which can assist the model in iterative learning within data-scarce scenarios. The prompt context must include function definitions, docstrings, capability descriptions, and chain-of-thought components that explain argument derivation and function selection logic. Hard-negative examples of similar but incorrect functions help improve the model's discrimination ability. The construction process leverages demonstration retrieval by scanning existing query-response pairs while maintaining a growing corpus of past use cases that enables system personalisation through live interactions. This approach incorporates multi-step refinement, from mining raw function calls to generating natural language queries with chain-of-thought reasoning. While these methods focus on improving function calling through demonstration and refinement, the Ask-when-Needed prompting strategy [155] enhances LLMs' ability to handle unclear instructions by encouraging proactive question-asking before API calls, achieving significant improvements in accuracy while maintaining efficiency when paired with GPT-4.

In multimodal context engineering scenarios, function calling presents enhanced complexity as LLMs must trigger functions utilizing visual, auditory, or alternative structured inputs beyond textual data. Contemporary developments in vision-language modeling have equipped multimodal agents with capabilities to comprehend and respond to heterogeneous input modalities, demanding innovative methodologies for context design and parameter extraction. Approaches for integrating visual data into function calling pipelines allow models to process both imagery and text for determining suitable function invocations. Multi-modal agent tuning methodologies [27] illustrate how VLMs function as controllers for tool usage reasoning, whereas frameworks such as MLLM-Tool [150] combine multimodal encoders with LLMs to interpret visual inputs and identify corresponding tools. These methodologies enable models to derive function parameters from visual content, including location identification from images or scene context comprehension for targeted API invocations. Integration architectures like VisionGPT [49] exemplify how to decompose tasks and orchestrate coordination among visual models, language models, and external tools, merging

multimodal information to accurately populate function parameters. This cross-modal reasoning capability broadens traditional text-based function calling to more diverse input scenarios. Assessment frameworks including VisualWebArena [51] measure multimodal agent performance on authentic visual web tasks, evaluating their tool usage abilities and visual environment interactions. These evaluation benchmarks illuminate both the capabilities and existing constraints of multimodal function calling systems.

These context engineering strategies effectively address **Challenge 2.1**, **Challenge 2.2**, **Challenge 3.2** and **Challenge 3.6** by improving function triggering accuracy and reducing hallucination through better-structured prompts and examples.

5.3 Function Generation (or Selection) Strategies

Once the prompt is constructed, the LLM generates functions under controlled output constraints. Using **context-free grammar (CFG)** [100, 121] restricts outputs to syntactically valid sequences, narrowing the token space, reducing invalid generations, and improving reliability and integration by enforcing grammar-conformant structures.

Beyond syntactic control, recent systems enhance function generation and tool use across affective, visual, retrieval, dialogue, and multi-agent settings: TOOL-ED [9] treats COMET-like knowledge as callable tools to improve affective responses; VisionMask [203] uses attribution to localize visual regions that trigger tool invocation; TR-Feedback [171] employs iterative LLM feedback to boost tool retrieval and establishes in-/out-of-domain benchmarks; APEC-Travel [46] performs interleaved, streaming dialogues to elicit preferences and achieve accurate, proactive, and credible function calling; IBSEN [32] coordinates director–actor agents for controlled script generation; PAE [201] combines proposer, agent, and evaluator to achieve large zero-shot gains on unseen tasks and sites; and Experiential Co-Learning [105] improves code-related function generation via agent cooperation and experience transfer. These approaches can improve the accuracy of LLMs in function calls, which demand high precision, thereby addressing **Challenge 3.2**, **Challenge 3.4** and **Challenge 3.5**.

5.4 Function Mapping Strategies

Function mapping plays a crucial role in deploying function calling, primarily responsible for transforming model outputs at the semantic level into executable commands in the physical space. Moreover, as shown in Figure 10, function mapping involves Pronoun Mapping, Format Alignment, and Error Checking.

5.4.1 Mapping of Pronouns. First, it is necessary to map some pronouns. For instance, when querying "the weather in our city today" the model output "GetWeather(date='Today', city='Our City')" can be recognized and converted to "GetWeather(date='20240101', city='Hangzhou')" This can be achieved through predefined human-designed rules (by constructing and maintaining necessary information mapping dictionaries) or by a small language model. Proper construction of the LLM data itself can also address this issue, but robust backups for industrial scenarios are very important. These conversions are typically achieved through simple mappings via human-designed rules [55, 72], knowledge structures reasoning [188], resolving and mapping via small language models [56], UI-guided token selection and interleaved streaming [68], or directly adding some specific tuning samples to finetune and enhance the LLM's direct resolving capabilities, thus addressing **Challenge 3.3** and **Challenge 4.3** to some extent.

5.4.2 Strict Format Alignment Mapping. Additionally, there is often a large gap between the user's natural language input and the strict requirements of function parameters (which change according to the function). For example, when querying the weather in "Hangzhou" on "January

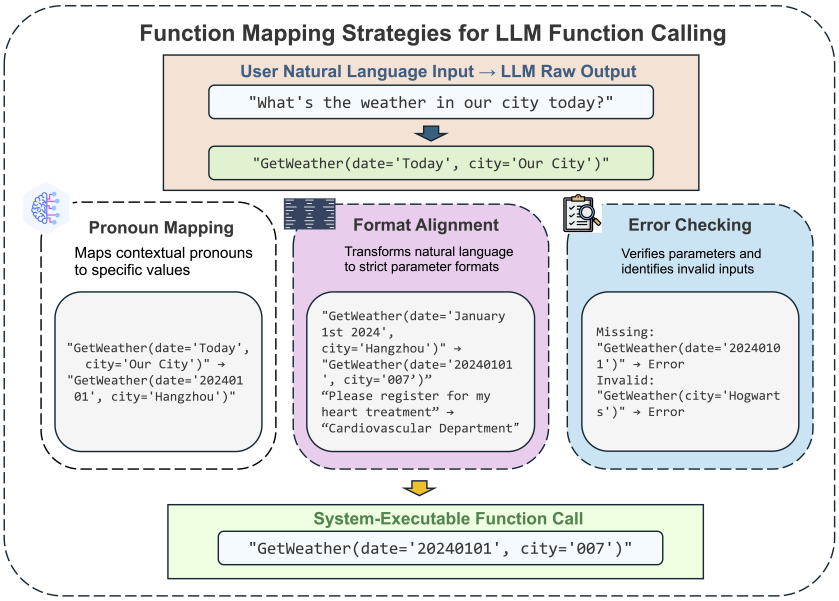

Fig. 10. Function mapping strategies in LLM function calling, illustrating the transformation process from natural language input to system-executable function calls through pronoun mapping, format alignment, and error checking.

1st, 2024," the model output "GetWeather(date='January 1st 2024', city='Hangzhou')" is converted to "GetWeather(date='20240101', city='007')" to meet actual system requirements. Or, when a user inputs "Please register for my heart treatment" at a hospital, it needs to be mapped to "Cardiovascular Department" This mapping can be maintained in the service through an information mapping dictionary, or by allowing the LLM to perform a first-step alignment through LLM CoT, or by training a small language classifier with semantic features to match function parameters. Syllabus [138] introduces a portable curriculum learning library that provides unified APIs and format alignment mechanisms for training RL agents across different environments and frameworks. These approaches also address **Challenge 4.3** to some extent.

5.4.3 Identification of Invalid Inputs. In addition, the mapping stage also needs to verify that all required parameters are included and assess the accuracy of enumerated values. This effectively addresses issues of missing or invalid parameters and mitigates the impact of **Challenge 3.1** at the system level. For example, if a model returns a query "GetWeather(date='20240101')" missing city information, the parameter checker will identify the missing city parameter and trigger a default response configured by operations, which is then returned to the user. Or if a query requests tomorrow's weather in "Hogwarts" with the output "GetWeather(date='20240101', city='Hogwarts')" and "Hogwarts" is not recognized in the city code table, the checker will flag it as an invalid value and trigger a default response for unsupported values, which is then communicated to the user. Moreover, function mapping could also involve filling in some parameters invisible to the LLM, such as fields related to permissions and UUIDs, which are supplemented during the backend processing stage to ensure the request's integrity and security. Function mapping fundamentally involves utilizing system-level rules and algorithms to ensure the robustness of the function-calling process and enhance the user experience in a way that is both reliable and intuitive. Effective function mapping addresses **Challenge 4.3** and can mitigate the impact of **Challenge 3.1** at the system level.

5.4.4 Privacy and Security Considerations. In the process of function mapping, systems inevitably process sensitive user data, presenting significant privacy and security challenges. These systems frequently handle personal information such as geographical location, time-related preferences, and user behavior patterns when converting natural language inputs into executable functions. Protecting such sensitive data throughout the entire mapping workflow represents a fundamental challenge for production deployments. Studies by Wu et al. [167] expose jailbreak attacks achieving success rates exceeding 90%, whereas metadata-based manipulation attacks [85] demonstrate the ability to trigger malicious tool invocations with 81–95% effectiveness through strategically crafted tool descriptions. In this situation, the gatekeeper frameworks [146] offer effective solutions by filtering sensitive content prior to processing, thereby safeguarding user privacy during function mapping operations. PrivacyChecker [154] propose a model-agnostic, contextual integrity based mitigation approach for reducing privacy leakage. These method partially addresses the privacy concern in **Challenge 3.3** and **Challenge 4.3**.

5.5 Response Generation Strategies

The simplest method for response generation involves initially using a model to generate placeholder results [35, 99, 122], which are then replaced with the outputs from an API call (either through direct matching or rewriting with a larger model). This approach ensures the model's planning and intent understanding are executed end-to-end. However, this method makes the function calls and their outputs unpredictable [179], and the execution of functions may lead to unexpected results.

5.5.1 Templates. Templates play a crucial role in structuring function calling outputs. Several works have demonstrated that structured templates significantly improve function calling accuracy and parameter selection [102, 109]. The template approach has proven especially valuable in API interaction scenarios where precise parameter formatting and type checking are crucial [57]. Studies indicate that well-designed templates can help models better understand function signatures and generate more accurate API calls [136]. These template-based approaches effectively address **Challenge 3.1** and **Challenge 4.3**.

5.5.2 Review. Response review mechanisms ensure reliable function execution through systematic validation. Parameter boundary checking and type validation during review phase have proven effective in preventing common function calling errors [109]. Beyond simple validation, interactive review and correction approaches have emerged - Song et al. implemented a schema-based parameter checking and response parsing system [134], while Shi et al. proposed a cooperative multi-agent framework that enables specialized agents to dynamically review and correct each other's actions [129]. A comprehensive evaluation by Jacovi et al. demonstrated that careful review mechanisms are crucial for tool-assisted systems, though they also found that existing review approaches still struggle with effective tool integration and validation [43]. Nathani et al. further advanced this field by introducing a multi-aspect feedback framework that integrates multiple specialized review modules to address different types of errors, showing significant improvements in reasoning accuracy [90]. Xu et al. proposed using compression and selective augmentation during review to make the process more efficient while maintaining effectiveness [170]. QueryAgent [39] proposes an environmental feedback-based framework for reliable and efficient function generation, introducing stepwise self-correction and targeted error guidance. These mechanisms are crucial for ensuring successful execution in complex API interactions. These review mechanisms effectively address **Challenge 3.1**, **Challenge 4.1**, and **Challenge 4.4**.

5.5.3 RAG. **Retrieval-Augmented Generation** (**RAG**) enhances function calling accuracy by leveraging existing examples [26]. By incorporating previously successful function calls, RAG

systems can better map user queries to appropriate API calls [11, 66, 91, 115, 117, 136, 184]. Nguyen et al. propose SFR-RAG [91], a contextually faithful LLM specifically optimized for retrieval-augmented generation, introducing a standardized evaluation framework and demonstrating strong performance in contextual understanding and citation abilities with significantly fewer parameters than existing solutions. Chan et al. [11] evaluate RAG for API integration by comparing semantic search approaches using different embedding models and analyzing the limitations of retrieval-based methods compared with fine-tuning based solutions. The NexusRaven [136] shows that carefully curated demonstration sets can significantly improve parameter selection and function matching. These enhancements help maintain a high level of clarity and consistency in responses, which is crucial for applications requiring high reliability and ease of use. These strategies not only optimize the interaction between users and AI systems but also contribute to creating a more efficient and user-friendly experience. These approaches can effectively address **Challenge 3.4**, **Challenge 3.5**, **Challenge 4.1** and **Challenge 4.2**.

5.6 Memory Scheme Strategy

Implementing an effective memory scheme in multi-turn dialogue is crucial for reliable function calls and parameter extraction. Based on observed gaps in contextual continuity, we propose maintaining extracted function parameters from the most recent K turns, with decay beyond K. For example, from "the weather in Hangzhou on 1st January 2024," the system extracts and caches GetWeather(date="20240101", city="Hangzhou"). On the follow-up "What about Xi'an?", it extracts only city="Xi'an", reuses the cached date, and executes GetWeather(date="tomorrow", city="Xi'an"). This design supports discrete parameter caching, selective reuse, and attenuation of stale entries to improve robustness in multi-iterative conversations.

Dynamic filtering and selective retention of context based on the current query are essential for improving dialogue quality and relevance by suppressing irrelevant information and maintaining focus on immediate needs. To address continuity and parameter consistency in multi-turn interactions, recent memory-augmented frameworks provide complementary designs: MemoryBank [199] introduces hierarchical storage, Ebbinghaus-inspired updates, and cross-conversation retrieval; SCM [64] employs a memory stream and controller for long-term maintenance; TiM [71] stores and recalls thoughts with pre-response recalling and post-response thinking; RET-LLM [86] uses a triplet read–write store with a controller and LSH-based indexing; SYNAPSE [197] leverages trajectory-as-exemplar prompting with state abstraction and similarity retrieval; METAAGENTS [62] enables collaborative agents with perception, memory, reasoning, and execution; TradingGPT [61] adopts a three-layer memory and inter-agent debate for financial decision-making; and DoraemonGPT [178] integrates task-related symbolic memory, spatio-temporal and knowledge tools, and MCTS-based planning. These approaches collectively strengthen long-horizon retention, retrieval, and planning, complementing lightweight dynamic filtering in practical systems.

These memory management strategies and memory-augmented frameworks effectively address **Challenge 3.6** by enabling more coherent and context-aware interactions across extended conversations. Additionally, by maintaining accurate parameter history and context, these approaches help mitigate **Challenge 3.1** and **Challenge 3.3** through improved parameter extraction and pronoun resolution in multi-turn dialogues.

5.7 Function Call Latency Strategies

The efficient execution of function calling relies on low latency, especially when managing long contexts. Although **key-value** (**KV**) cache is common for long contexts, it can negatively impact performance in function calling scenarios. The vLLM [149] framework introduces PagedAttention [54], enabling efficient attention computation through page-based memory organization and

significantly reducing memory requirements. Combined with continuous batching, vLLM achieves higher throughput while maintaining low latency. FlashAttention [16, 17, 125] presents an IO-aware attention algorithm to reduce memory access, while SpecInfer [82] enhances efficiency through speculative inference. For multiple function calls, system-level planning is essential. LLM Compiler [50, 132] effectively plans and executes calls in parallel using a **Directed Acyclic Graph** (**DAG**) to manage dependencies, significantly reducing latency in complex scenarios and improving throughput. For further optimization, several techniques have proven effective. Techniques such as model pruning [78, 88, 140] and knowledge distillation [172, 174] achieve significant latency reductions while preserving function calling accuracy and stability. Quantization methods [19, 147] can further optimize inference speeds in production environments with minimal quality degradation.

Notably, smaller models have also demonstrated powerful function calling capabilities. ThorV2 [7] demonstrates that smaller models can possess enhanced function calling capabilities, even outperforming larger commercial LLMs on specific CRM operations. Meanwhile, TinyAgent [20], through efficient model compression and optimization techniques, shows that small language models can run entirely on local hardware (like a MacBook) and achieve performance comparable to GPT-4-Turbo, marking a significant advancement in edge-based function calling capabilities. By maintaining accurate parameter history and context, these approaches help mitigate **Challenge 3.4** and **Challenge 3.5** through improved model efficiency for real-time applications and robust orchestration of parallel tasks.

6 Looking Ahead: Open Issues and Discussions for Function Calling in LLMs

Despite the significant progress in function calling capabilities for LLMs, several critical open issues remain to be addressed. These issues span various aspects of function calling systems, from fundamental service-level issues to practical concerns about usability, optimization, and function isolation.

6.1 Open Issue 1: Service Quality, Function Usability, and Feedback in Function Calling

Function calling in real-world services requires an integrated perspective that spans evaluation, usability, and feedback. Current assessments, as summarized in Appendix Sec. A, emphasize technical capability but underrepresent service requirements such as end-to-end latency, throughput in tool-augmented reasoning, user experience, and security; for example, LLM plugins can be slower than traditional systems and multi-stage workflows exacerbate delays, particularly in Agent Laboratory [123]. Security concerns are significant, as jailbreak function attacks have achieved high success rates across major models, including GPT-4 [94], Claude-3.5-Sonnet [4], and Gemini-1.5-Pro [143], as reported by Wu et al. [167]. Practical effectiveness also depends on the usability and modifiability of functions, yet integration is often hindered by data interoperability gaps, architectural constraints, and coupling with legacy systems, which impedes deployment flexibility and reduces training efficacy. Standardized API modification frameworks with clear evaluation protocols, design specifications, validation methods, and deployment guidelines, together with modular design, can reduce barriers and improve adaptability. Finally, feedback-driven optimization remains limited in ambiguous settings, since RLHF and supervised methods struggle with unstructured feedback and may propagate errors in multi-step pipelines. A unified framework is needed that jointly measures latency, accuracy, user satisfaction, and security, standardizes function design and modification, and systematically incorporates robust, uncertainty-aware, and interactive feedback mechanisms.

6.2 Open Issue 2: Function Isolation and Post-Processing Strategies

Implementing appropriate isolation mechanisms and post-processing frameworks for distinct functional components is imperative to meet commercial requirements and regulatory compliance

specifications. These implementations necessitate flexible design methodologies and elevated customization capabilities.

Examples include microservice architectures, wherein individual functional components are deployed as autonomous service entities with precisely delineated interfaces. This approach facilitates enhanced control over operational behavior and simplifies regulatory compliance by isolating sensitive processes. Additionally, middleware layers designed explicitly for post-processing tasks ensure that computational outputs conform to the necessary specifications.

A primary challenge is ensuring that isolation strategies and post-processing frameworks maintain effectiveness across heterogeneous implementation scenarios, while satisfying regulatory requirements and preserving service efficiency. Potential solutions might include adaptive algorithms that dynamically reconfigure API characteristics based on real-time analytics, policy-driven management systems that facilitate automated compliance mechanisms, and machine learning models for monitoring performance metrics to enable resource allocation adjustments.

6.3 Open Issue 3: Bridging Function Calling between LLMs and Conventional Industrial Systems

Traditional industrial systems, such as search engines and dialog management frameworks, have long relied on deterministic pipelines composed of retrieval, ranking, and response-generation modules. Each component performs a narrowly defined function, and the overall performance depends on manually engineered features. This modular design ensures interpretability, scalability, and compliance, but it also restricts flexibility and cross-task generalization.

The emergence of LLMs has begun to reshape this paradigm. Instead of maintaining multiple specialized subsystems, LLMs can integrate retrieval, reasoning, and generation into a unified agent process, such as the search-r1 [47] agent, to utilize LLM reasoning abilities to decompose the query for more precise search.

Within this evolving landscape, function calling represents a key mechanism for bridging LLM-based agent systems and practical industry systems. For example, company such as Baidu propose the AI search paradiam [60]. It provides a structured function call through which LLMs can interact with external tools [77], enforce operation boundaries, and extend their capabilities to real-world applications. As LLMs continue to replace conventional components in search and dialog systems, exploring efficient, interpretable, and controllable function-calling frameworks will become a crucial direction for both research and industrial deployment.

6.4 Open Issue 4: Standardization and Design of Callable Functions for LLM Integration

A critical yet underexplored dimension in current research on function calling lies in the design and standardization of the existing functions. Most existing efforts concentrate on handmade LLM adaptive fuctions, while overlookin how to standlize traditional fuction into LLM era. The absence of clear design standards may hinder the seamless integration of LLMs into existing industrial systems and application scenarios.

In traditional software and service-oriented architectures, function specifications follow explicit interface definitions such as Java Spring schemas. These standards ensure clarity in parameter types, exception handling, and response structures, enabling automated integration and compliance verification. However, when extending to LLM-driven systems, functions must additionally support natural-language alignment, flexible parameter mapping, and contextual adaptation. The absence of unified conventions for these aspects limits interoperability across tool collections and complicates large-scale orchestration of multi-step reasoning tasks.

Future progress will depend on establishing unified design principles that bridge traditional software specifications and LLM-driven adaptability. Such standardization would enable callable

functions to become not only technically interoperable but also semantically aligned with the reasoning capabilities of large language models.

7 Conclusion

This comprehensive survey has examined function calling in LLMs from an industrial perspective, systematically analyzing the challenges, solutions, and future directions in this rapidly evolving field. We have outlined a complete function calling pipeline consisting of three critical stages: pre-call, on-call, and post-call, and detailed the specific challenges encountered at each stage. The key challenges include intent recognition, function redundancy, parameter extraction, function hallucination, and multi-call procedures. Our review and discussion demonstrated that function calling significantly enhances LLMs' capabilities by enabling structured outputs and real-time interactions with external systems. As the field continues to evolve, we anticipate the development of more sophisticated techniques for context management, complex multi-function calls, and real-time parameter validation, leading to more powerful and practical AI systems that can better serve human needs while maintaining reliability and efficiency.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv:2303.08774* (2023).

[2] Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)* 51, 4 (2018), 1–37.

[3] Daniel Andor, Luheng He, Kenton Lee, and Emily Pitler. 2019. Giving BERT a calculator: Finding operations and arguments with reading comprehension. *arXiv:1909.00109* (2019).

[4] Anthropic. 2024. *The Claude 3 Model Family: Opus, Sonnet, Haiku.* Model Card. Anthropic. Retrieved from https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf

[5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv:2309.16609* (2023).

[6] Kinjal Basu, Ibrahim Abdelaziz, Subhajit Chaudhury, Soham Dan, Maxwell Crouse, Asim Munawar, Sadhana Kumaravel, Vinod Muthusamy, Pavan Kapanipathi, et al. 2024. API-BLEND: A comprehensive corpora for training and benchmarking API LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12859–12870.

[7] Nirav Bhan, Shival Gupta, Sai Manaswini, Ritik Baba, Narun Yadav, Hillori Desai, Yash Choudhary, Aman Pawar, Sarthak Shrivastava, and Sudipta Biswas. 2024. Benchmarking floworks against OpenAI & Anthropic: A novel framework for enhanced LLM function calling. *arXiv:2410.17950* (2024).

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.

[9] Huiying Cao, Yiqun Zhang, Shi Feng, Xiaocui Yang, Daling Wang, and Yifei Zhang. 2025. TOOL-ED: Enhancing empathetic response generation with the tool calling capability of LLM. In *Proceedings of the 31st International Conference on Computational Linguistics*. 5305–5320.

[10] Shuxiang Cao, Zijian Zhang, Mohammed Alghadeer, Simone D Fasciati, Michele Piscitelli, Mustafa Bakr, Peter Leek, and Alán Aspuru-Guzik. 2024. Agents for self-driving laboratories applied to quantum computing. *arXiv:2412.07978* (2024).

[11] Robin Chan, Katsiaryna Mirylenka, Thomas Gschwind, Christoph Miksovic, Paolo Scotton, Enrico Toniato, and Abdel Labbi. 2024. Adapting LLMs for structured natural language API integration. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 991–1000.

[12] Mingyang Chen, Haoze Sun, Tianpeng Li, Fan Yang, Hao Liang, Keer Lu, Bin Cui, Wentao Zhang, Zenan Zhou, and Weipeng Chen. 2024. Facilitating multi-turn function calling for LLMs via compositional instruction tuning. *arXiv:2410.12952* (2024).

[13] Yuhan Chen, Ang Lv, Ting-En Lin, Changyu Chen, Yuchuan Wu, Fei Huang, Yongbin Li, and Rui Yan. 2024. Fortify the shortest stave in attention: Enhancing context awareness of large language models for effective tool use. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11160–11174.

[14] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 4302–4310.

[15] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2924–2936.

[16] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691* (2023).

[17] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.

[18] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General Language Model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 320–335.

[19] Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. 2024. Exploiting LLM quantization. In *Advances in Neural Information Processing Systems* 37 (2024), 41709–41732.

[20] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2024. Tinyagent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 80–88.

[21] Wenqi Fan, Pangjing Wu, Yujuan Ding, Liangbo Ning, Shijie Wang, and Qing Li. 2025. Towards retrieval-augmented large language models: Data management and system design. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 4509–4512.

[22] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. 2025. Retool: Reinforcement learning for strategic tool use in llms. *arXiv:2504.11536* (2025).

[23] Michael Fore, Simranjit Singh, and Dimitrios Stamoulis. 2024. GeckOpt: LLM system efficiency via intent-based tool selection. In *Proceedings of the Great Lakes Symposium on VLSI 2024*. 353–354.

[24] Silin Gao, Jane Dwivedi-Yu, Ping Yu, Xiaoqing Ellen Tan, Ramakanth Pasunuru, Olga Golovneva, Koustuv Sinha, Asli Celikyilmaz, Antoine Bosselut, and Tianlu Wang. 2024. Efficient tool use with chain-of-abstraction reasoning. In *Proceedings of the 31st International Conference on Computational Linguistics*. 2727–2743.

[25] Shen Gao, Zhengliang Shi, Minghang Zhu, Bowen Fang, Xin Xin, Pengjie Ren, Zhumin Chen, Jun Ma, and Zhaochun Ren. 2024. Confucius: Iterative tool learning from introspection feedback by easy-to-difficult curriculum. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18030–18038.

[26] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997* 2, 1 (2023).

[27] Zhi Gao, Bofei Zhang, Pengxiang Li, Xiaojian Ma, Tao Yuan, Yue Fan, Yuwei Wu, Yunde Jia, Song-Chun Zhu, and Qing Li. 2024. Multi-modal agent tuning: Building a vlm-driven agent for efficient tool usage. *arXiv:2412.15606* (2024).

[28] Yingqiang Ge, Yujie Ren, Wenyue Hua, Shuyuan Xu, Juntao Tan, and Yongfeng Zhang. 2023. Llm as os (llmao), agents as apps: Envisioning aios, agents and the aios-agent ecosystem. *arXiv:2312.03815* (2023).

[29] GlaiveAI. 2024. Glaive Function Calling v2 Dataset. Retrieved from https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2

[30] Jian Guan, Wei Wu, Peng Xu, Hongning Wang, Minlie Huang, et al. 2024. AMOR: A recipe for building adaptable modular knowledge agents through process feedback. In *Advances in Neural Information Processing Systems* 37 (2024), 126118–126148.

[31] Anchun Gui, Jian Li, Yong Dai, Nan Du, and Han Xiao. 2024. Look before you leap: Towards decision-aware and generalizable tool-usage for large language models. *arXiv:2402.16696* (2024).

[32] Senyu Han, Lu Chen, Li-Min Lin, Zhengshan Xu, and Kai Yu. 2024. IBSEN: Director-actor agent collaboration for controllable and interactive drama script generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1607–1619.

[33] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Yicheng Chen, Cunyin Peng, Jinjie Gu, and Chenyi Zhuang. 2025. Exploring superior function calls via reinforcement learning. *arXiv e-prints* (2025), arXiv–2508.

[34] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Cunyin Peng, Yicheng Chen, Xiangyu Zhao, Jinjie Gu, and Chenyi Zhuang. 2025. FunReason: Enhancing large language models' function calling via self-refinement multiscale loss and automated data refinement. *arXiv:2505.20192* (2025).

[35] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. In *Advances in Neural Information Processing Systems* 36 (2023), 45870–45894.

[36] Yupu Hao, Pengfei Cao, Zhuoran Jin, Huanxuan Liao, Yubo Chen, Kang Liu, and Jun Zhao. 2025. CITI: Enhancing tool utilizing ability in large language models without sacrificing general performance. In *Proceedings of the AAAI Conference on Artificial Intelligence* 39 (2025), 23996–24004.

[37] Jáchym Herynek and Stefan Edelkamp. 2024. Heuristic planner for communication-constrained multi-agent multi-goal path planning. *arXiv:2412.13719* (2024).
[38] Tenghao Huang, Dongwon Jung, Vaibhav Kumar, Mohammad Kachuee, Xiang Li, Puyang Xu, and Muhao Chen. 2024. Planning and editing what you retrieve for enhanced tool learning. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 975–988.
[39] Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. 2024. QueryAgent: A reliable and efficient reasoning framework with environmental feedback based self-correction. *arXiv:2403.11886* (2024).
[40] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of LLM agents: A survey. *arXiv:2402.02716* (2024).
[41] Zhongzhen Huang, Kui Xue, Yongqi Fan, Linjie Mu, Ruoyu Liu, Tong Ruan, Shaoting Zhang, and Xiaofan Zhang. 2024. Tool calling: Enhancing medication consultation via retrieval-augmented large language models. *arXiv:2404.17897* (2024).
[42] Wolfram Research, Inc. [n. d.]. Mathematica, Version 14.0. Retrieved from https://www.wolfram.com/mathematica. Champaign, IL, 2024.
[43] Alon Jacovi, Avi Caciularu, Jonathan Herzig, Roee Aharoni, Bernd Bohnet, and Mor Geva. 2023. A comprehensive evaluation of tool-assisted generation strategies. *arXiv:2310.10062* (2023).
[44] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
[45] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. 2025. Verltool: Towards holistic agentic reinforcement learning with tool use. *arXiv:2509.01055* (2025).
[46] Song Jiang, Da JU, Andrew Cohen, Sasha Mitts, Aaron Foss, Justine T. Kao, Xian Li, and Yuandong Tian. 2024. Towards full delegation: Designing ideal agentic behaviors for travel planning. *arXiv:2411.13904* (2024).
[47] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv:2503.09516* (2025).
[48] Ulas Berk Karli, Juo-Tung Chen, Victor Nikhil Antony, and Chien-Ming Huang. 2024. Alchemist: LLM-aided end-user development of robot applications. In *Proceedings of the 2024 ACM/IEEE International Conference on Human–Robot Interaction*. 361–370.
[49] Chris Kelly, Luhui Hu, Bang Yang, Yu Tian, Deshun Yang, Cindy Yang, Zaoshan Huang, Zihao Li, Jiayin Hu, and Yuexian Zou. 2024. Visiongpt: Vision-language understanding agent using generalized multimodal framework. *arXiv:2403.09027* (2024).
[50] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM compiler for parallel function calling. In *International Conference on Machine Learning*. PMLR, 24370–24391.
[51] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 881–905.
[52] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems* 35 (2022), 22199–22213.
[53] Yilun Kong, Jingqing Ruan, Yihong Chen, Bin Zhang, Tianpeng Bao, Shi Shiwei, Xiaoru Hu, Hangyu Mao, Ziyue Li, Xingyu Zeng, et al. 2024. Tptu-v2: Boosting task planning and tool usage of large language model-based agents in real-world industry systems. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: IndustryTtrack*. 371–385.
[54] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
[55] Heeyoung Lee, Angel Chang, Yves Peirsman, Nathanael Chambers, Mihai Surdeanu, and Dan Jurafsky. 2013. Deterministic coreference resolution based on entity-centric, precision-ranked rules. *Computational Linguistics* 39, 4 (2013), 885–916.
[56] Kenton Lee, Luheng He, Mike Lewis, and Luke Zettlemoyer. 2017. End-to-end neural coreference resolution. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. 188–197.
[57] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.
[58] Lei Li, Yekun Chai, Shuohuan Wang, Yu Sun, Hao Tian, Ningyu Zhang, and Hua Wu. 2023. Tool-augmented reward modeling. *arXiv:2310.01045* (2023).
[59] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A benchmark for tool-augmented llms. *arXiv:2304.08244* (2023).

[60] Yuchen Li, Hengyi Cai, Rui Kong, Xinran Chen, Jiamin Chen, Jun Yang, Haojie Zhang, Jiayi Li, Jiayi Wu, Yiqun Chen, et al. 2025. Towards AI search paradigm. *arXiv:2506.17188* (2025).
[61] Yang Li, Yangyang Yu, Haohang Li, Zhi Chen, and Khaldoun Khashanah. 2023. TradingGPT: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance. *arXiv:2309.03736* (2023).
[62] Yuan Li, Yixuan Zhang, and Lichao Sun. 2023. Metaagents: Simulating interactions of human behaviors for llm-based task-oriented coordination via collaborative generative agents. *arXiv:2310.06500* (2023).
[63] Zeyang Li and Navid Azizan. 2024. Safe multi-agent reinforcement learning with convergence to generalized nash equilibrium. *arXiv:2411.15036* (2024).
[64] Xinnian Liang, Bing Wang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun Li. 2023. Unleashing infinite-length input capacity for large-scale language models with self-controlled memory system. *arXiv e-prints* (2023), arXiv–2304.
[65] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. 2024. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *Intelligent Computing* 3 (2024), 0063.
[66] John Licato, Logan Fields, Stephen Steinle, and Brayden Hollis. 2024. Shot selection to determine legality of actions in a rule-heavy environment. In *2024 IEEE Conference on Games (CoG)*. IEEE, 1–8.
[67] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
[68] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for GUI visual agent. 19498–19508.
[69] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, et al. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv:2410.04587* (2024).
[70] Jiacheng Liu, Skyler Hallinan, Ximing Lu, Pengfei He, Sean Welleck, Hannaneh Hajishirzi, and Yejin Choi. 2022. Rainier: Reinforced knowledge introspector for commonsense question answering. *arXiv:2210.03078* (2022).
[71] Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. 2023. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv:2311.08719* (2023).
[72] Ruicheng Liu, Rui Mao, Anh Tuan Luu, and Erik Cambria. 2023. A brief survey on recent advances in coreference resolution. *Artificial Intelligence Review* 56, 12 (2023), 14439–14481.
[73] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, et al. 2024. Toolace: Winning the points of llm function calling. *arXiv:2409.00920* (2024).
[74] Xingyu Bruce Liu, Shitao Fang, Weiyan Shi, Chien-Sheng Wu, Takeo Igarashi, and XiangAnthony' Chen. 2024. Proactive conversational agents with inner thoughts. *arXiv:2501.00383* (2024).
[75] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, et al. 2024. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. In *Advances in Neural Information Processing Systems* 37 (2024), 54463–54482.
[76] Zhaoyang Liu, Zeqiang Lai, Zhangwei Gao, Erfei Cui, Zhiheng Li, Xizhou Zhu, Lewei Lu, Qifeng Chen, Yu Qiao, Jifeng Dai, et al. 2024. Controlllm: Augment language models with tools by searching on graphs. In *European Conference on Computer Vision*. Springer, 89–105.
[77] Rui Lu, Zhenyu Hou, Zihan Wang, Hanchen Zhang, Xiao Liu, Yujiang Li, Shi Feng, Jie Tang, and Yuxiao Dong. 2025. Deepdive: Advancing deep search agents with knowledge graphs and multi-turn rl. *arXiv:2509.10446* (2025).
[78] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems* 36 (2023), 21702–21720.
[79] Zixian Ma, Jianguo Zhang, Zhiwei Liu, Jieyu Zhang, Juntao Tan, Manli Shu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Caiming Xiong, et al. 2024. TACO: Learning multi-modal action models with synthetic chains-of-thought-and-action. *arXiv:2412.05479* (2024).
[80] Graziano A. Manduzio, Federico A. Galatolo, Mario GCA Cimino, Enzo Pasquale Scilingo, and Lorenzo Cominelli. 2024. Improving small-scale large language models function calling for reasoning tasks. *arXiv:2410.18890* (2024).
[81] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. 2023. Augmented language models: A survey. *arXiv:2302.07842* (2023).
[82] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.
[83] Antonio Valerio Miceli Barone and Rico Sennrich. 2017. A parallel corpus of python functions and documentation strings for automated code documentation and code generation. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing* 2 (2017), 314–319.

[84] Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Codas, Clarisse Simoes, Sahaj Agarwal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, et al. 2023. Orca 2: Teaching small language models how to reason. *arXiv:2311.11045* (2023).

[85] Kanghua Mo, Li Hu, Yucheng Long, and Zhihao Li. 2025. Attractive metadata attack: Inducing LLM agents to invoke malicious tools. *arXiv:2508.02110* (2025).

[86] Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. 2023. Ret-llm: Towards a general read-write memory for large language models. *arXiv:2305.14322* (2023).

[87] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. Orca: Progressive learning from complex explanation traces of gpt-4. *arXiv:2306.02707* (2023).

[88] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Bhuminand Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Compact language models via pruning and knowledge distillation. In *Advances in Neural Information Processing Systems* 37 (2024), 41076–41102.

[89] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. Webgpt: Browser-assisted question-answering with human feedback. *arXiv:2112.09332* (2021).

[90] Deepak Nathani, David Wang, Liangming Pan, and William Yang Wang. 2023. MAF: Multi-aspect feedback for improving reasoning in large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 6591–6616.

[91] Xuan-Phi Nguyen, Shrey Pandit, Senthil Purushwalkam, Austin Xu, Hailin Chen, Yifei Ming, Zixuan Ke, Silvio Savarese, Caiming Xong, and Shafiq Joty. 2024. Sfr-rag: Towards contextually faithful llms. *arXiv:2409.09916* (2024).

[92] Kaiwen Ning, Jiachi Chen, Jingwen Zhang, Wei Lia, Zexu Wang, Yuming Feng, Weizhe Zhang, and Zibin Zheng. 2024. Defining and detecting the defects of the large language model-based autonomous agents. *arXiv:2412.18371* (2024).

[93] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. 2021. Show your work: Scratchpads for intermediate computation with language models. *arXiv:2112.00114* (2021).

[94] OpenAI. 2023. ChatGPT Plugins. Retrieved fromhttps://openai.com/blog/chatgpt-plugins Accessed: 2024-07-18.

[95] OpenAI. 2023. Introducing Function Calling in ChatGPT. Retrieved from https://openai.com/blog/function-calling. Accessed: 2023-10-01.

[96] Yasser Otiefy and Alaa Alhamzeh. 2024. Exploring large language models in financial argument relation identification. In *Proceedings of the Joint Workshop of the 7th Financial Technology and Natural Language Processing, the 5th Knowledge Discovery from Unstructured Data in Financial Services, and the 4th Workshop on Economics and Natural Language Processing@ LREC-COLING 2024*. 119–129.

[97] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.

[98] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. Memgpt: Towards llms as operating systems. *arXiv:2310.08560* (2023).

[99] Aaron Parisi, Yao Zhao, and Noah Fiedel. 2022. Talm: Tool augmented language models. *arXiv:2205.12255* (2022).

[100] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-aligned decoding. In *Advances in Neural Information Processing Systems* 37 (2024), 24547–24568.

[101] Shishir Patil et al. 2024. Berkeley Function Calling Leaderboard (BFCL). Retrieved from https://github.com/ShishirPatil/gorilla/tree/main/berkeley-function-call-leaderboard

[102] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems* 37 (2024), 126544–126565.

[103] Danilo Peixoto. 2024. Pandora Tool-Calling Dataset. Retrieved from https://huggingface.co/datasets/danilopeixoto/pandora-tool-calling

[104] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. 2025. Toolrl: Reward is all tool learning needs. *arXiv:2504.13958* (2025).

[105] Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, YiFei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, et al. 2024. Experiential co-learning of software-developing agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5628–5640.

[106] Cheng Qian, Chenyan Xiong, Zhenghao Liu, and Zhiyuan Liu. 2024. Toolink: Linking toolkit creation and using through chain-of-solving on open-source model. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 831–854.

[107] Shuofei Qiao, Yixin Ou, Ningyu Zhang, Xiang Chen, Yunzhi Yao, Shumin Deng, Chuanqi Tan, Fei Huang, and Huajun Chen. 2023. Reasoning with language model prompting: A survey. In *Proceedings of the 61st annual meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5368–5393.

[108] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, et al. 2024. Tool learning with foundation models. *Comput. Surveys* 57, 4 (2024), 1–40.

[109] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv:2307.16789* (2023).

[110] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2024. Towards completeness-oriented tool retrieval for large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1930–1940.

[111] Changle Qu, Sunhao Dai, XiaochiWei, Hengyi Cai, ShuaiqiangWang, Dawei Yin, Jun Xu, and Ji-RongWen. 2025. Tool learning with large language models: A survey. *Frontiers of Computer Science* 19, 8 (2025), 198343.

[112] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. 2021. Scaling language models: Methods, analysis & insights from training gopher. *arXiv:2112.11446* (2021).

[113] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems* 36 (2024), 53728–53741.

[114] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.

[115] Gentiana Rashiti, Geethan Karunaratne, Mrinmaya Sachan, Abu Sebastian, and Abbas Rahimi. 2024. Retro-li: Small-scale retrieval augmented generation supporting noisy similarity searches and domain shift generalization. *arXiv:2410.00004* (2024).

[116] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[117] Kaushik Roy, Yuxin Zi, Chathurangi Shyalika, Renjith Prasad, Sidhaarth Murali, Vedant Palit, and Amit Sheth. 2024. QA-RAG: Leveraging question and answer-based retrieved chunk re-formatting for improving response quality during retrieval-augmented generation. (2024).

[118] Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, Tianpeng Bao, Hangyu Mao, Ziyue Li, Xingyu Zeng, Rui Zhao, et al. 2023. Tptu: Task planning and tool usage of large language model-based AI agents. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*.

[119] Avirup Saha, Lakshmi Mandal, Balaji Ganesan, Sambit Ghosh, Renuka Sindhgatta, Carlos Eberhardt, Dan Debrunner, and Sameep Mehta. 2024. Sequential API function calling using GraphQL schema. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 19452–19458.

[120] Sagar Srinivas Sakhinana, Geethan Sannidhi, and Venkataramana Runkana. 2024. Retrieval-augmented instruction tuning for automated process engineering calculations: A tool-chaining problem-solving framework with attributable reflection. *arXiv:2408.15866* (2024).

[121] Laboni Sarker, Mara Downing, Achintya Desai, and Tevfik Bultan. 2024. Syntactic robustness for llm-based code generation. *arXiv preprint arXiv:2404.01535* (2024).

[122] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.

[123] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. 2025. Agent laboratory: Using llm agents as research assistants. *Findings of the Association for Computational Linguistics (EMNLP'25)*. 5977–6043.

[124] Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get to the point: Summarization with pointer-generator networks. *arXiv:1704.04368* (2017).

[125] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.

[126] ChengAo Shen, Zhengzhang Chen, Dongsheng Luo, Dongkuan Xu, Haifeng Chen, and Jingchao Ni. 2025. Exploring multi-modal data with tool-augmented LLM agents for precise causal discovery. In *Findings of the Association for Computational Linguistics (ACL'25)*. 636–660.

[127] Weizhou Shen, Chenliang Li, Hongzhan Chen, Ming Yan, Xiaojun Quan, Hehong Chen, Ji Zhang, and Fei Huang. 2024. Small llms are weak tool learners: A multi-llm agent. *arXiv:2401.07324* (2024).

[128] Yuchen Shi, Siqi Cai, Zihan Xu, Yuei Qin, Gang Li, Hang Shao, Jiawei Chen, Deqing Yang, Ke Li, and Xing Sun. 2025. FlowAgent: Achieving compliance and flexibility for workflow agents. *arXiv:2502.14345* (2025).

[129] Zhengliang Shi, Shen Gao, Xiuyi Chen, Yue Feng, Lingyong Yan, Haibo Shi, Dawei Yin, Pengjie Ren, Suzan Verberne, and Zhaochun Ren. 2024. Learning to use tools via cooperative and interactive agents. *arXiv:2403.03031* (2024).

[130] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 11523–11530.

[131] Simranjit Singh, Michael Fore, Andreas Karatzas, Chaehong Lee, Yanan Jian, Longfei Shangguan, Fuxun Yu, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2024. LLM-dcache: improving tool-augmented LLMs with GPT-driven localized data caching. 1–4.

[132] Simranjit Singh, Andreas Karatzas, Michael Fore, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2024. An LLM-tool compiler for fused parallel function calling. *arXiv:2405.17438* (2024).

[133] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. 2023. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2998–3009.

[134] Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, et al. 2023. RestGPT: Connecting large language models with real-world RESTful APIs. *arXiv:2306.06624* (2023).

[135] Sakhinana Sagar Srinivas, Vijay Sri Vaikunth, and Venkataramana Runkana. 2024. Knowledge graph modeling-driven large language model operating system (LLM OS) for task automation in process engineering problem-solving. *arXiv:2408.14494* (2024).

[136] Venkat Krishna Srinivasan, Zhen Dong, Banghua Zhu, Brian Yu, Damon Mosk-Aoyama, Kurt Keutzer, Jiantao Jiao, and Jian Zhang. 2023. Nexusraven: A commercially-permissive language model for function calling. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*.

[137] Gita Sukthankar, Christopher Geib, Hung Hai Bui, David Pynadath, and Robert P. Goldman. 2014. *Plan, Activity, and Intent Recognition: Theory and Practice*. Newnes.

[138] Ryan Sullivan, Ryan Pégoud, Ameen Ur Rahmen, Xinchen Yang, Junyun Huang, Aayush Verma, Nistha Mitra, and John P. Dickerson. 2024. Syllabus: Portable curricula for reinforcement learning agents. *arXiv:2411.11318* (2024).

[139] Jiankai Sun, Chuanyang Zheng, Enze Xie, Zhengying Liu, Ruihang Chu, Jianing Qiu, Jiaqi Xu, Mingyu Ding, Hongyang Li, Mengzhe Geng, et al. 2025. A survey of reasoning with foundation models: Concepts, methodologies, and outlook. *Comput. Surveys* 57, 11 (2025), 1–43.

[140] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv:2306.11695* (2023).

[141] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. 2025. OS-genesis: Automating GUI agent trajectory construction via reverse task synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* 1 (2025), 5555–5579.

[142] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A large language model for science. *arXiv:2211.09085* (2022).

[143] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv:2403.05530* (2024).

[144] Qwen Team et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* 2, 3 (2024).

[145] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv:2302.13971* (2023).

[146] GodsGift Uzor, Hasan Al-Qudah, Ynes Ineza, and Abdul Serwadda. 2025. Guarding your conversations: Privacy gatekeepers for secure interactions with cloud-based AI models. In *2025 19th International Conference on Semantic Computing (ICSC)*. IEEE Computer Society, 235–242.

[147] Mart van Baalen, Andrey Kuzmin, Markus Nagel, Peter Couperus, Cedric Bastoul, Eric Mahurin, Tijmen Blankevoort, and Paul Whatmough. 2024. Gptvq: The blessing of dimensionality for llm quantization. *arXiv:2402.15319* (2024).

[148] Ashish Vaswani et al. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*. 5998–6008.

[149] vLLM Contributors. 2024. Easy, fast, and cheap LLM serving for everyone |. Retrieved from https://github.com/vllm-project/vllm.

[150] Chenyu Wang, Weixin Luo, Sixun Dong, Xiaohua Xuan, Zhengxin Li, Lin Ma, and Shenghua Gao. 2025. Mllm-tool: A multimodal large language model for tool agent learning. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 6678–6687.

[151] Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. 2025. Otc: Optimal tool calls via reinforcement learning. *arXiv e-prints* (2025), arXiv–2504.

[152] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.

[153] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2024. ToolGen: Unified tool retrieval and calling via generation. *arXiv:2410.03439* (2024).

[154] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. 2025. Privacy in action: Towards realistic privacy mitigation and evaluation for LLM-powered agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 17055–17074.

[155] Wenxuan Wang, Juluan Shi, Chaozheng Wang, Cheryl Lee, Youliang Yuan, Jen-tse Huang, and Michael R. Lyu. 2024. Learning to ask: When LLMs meet unclear instruction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 21784–21795.

[156] Xin Wang, Yuwei Zhou, Hong Chen, and Wenwu Zhu. 2024. Curriculum learning: Theories, approaches, applications, tools, and future directions in the era of large language models. In *Companion Proceedings of the ACM on Web Conference 2024*. 1306–1310.

[157] Yuanchun Wang, Jifan Yu, Zijun Yao, Jing Zhang, Yuyang Xie, Shangqing Tu, Yiyang Fu, Youhe Feng, Jinkai Zhang, Jingyao Zhang, et al. 2025. SoAy: A solution-based LLM API-using methodology for academic information seeking. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining* 1 (2025), 2660–2671.

[158] Yiqin Wang, Haoji Zhang, Jingqi Tian, and Yansong Tang. 2025. Ponder & press: Advancing visual GUI agent towards general computer control. In *Findings of the Association for Computational Linguistics (ACL'25)*. 1461–1473.

[159] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Shawn Ma, and Yitao Liang. 2023. Describe, explain, plan and select: Interactive planning with LLMs enables open-world multi-task agents. In *Advances in Neural Information Processing Systems* 36 (2023), 34153–34189.

[160] Zhiruo Wang, Zhoujun Cheng, Hao Zhu, Daniel Fried, and Graham Neubig. 2024. What are tools anyway? a survey from the language model perspective. *arXiv:2403.15452* (2024).

[161] Zekun Wang, Ge Zhang, Kexin Yang, Ning Shi, Wangchunshu Zhou, Shaochun Hao, Guangzheng Xiong, Yizhi Li, Mong Yuan Sim, Xiuying Chen, et al. 2023. Interactive natural language processing. *arXiv:2305.13246* (2023).

[162] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[163] Fangzhou Wu, Ning Zhang, Somesh Jha, Patrick McDaniel, and Chaowei Xiao. 2024. A new era in LLM security: Exploring security concerns in real-world LLM-based systems. *arXiv:2402.18649* (2024).

[164] Mengsong Wu, Tong Zhu, Han Han, Chuanyuan Tan, Xiang Zhang, and Wenliang Chen. 2024. Seal-tools: Self-instruct tool learning dataset for agent tuning and detailed benchmark. In *CCF International Conference on Natural Language Processing and Chinese Computing*. Springer. 372–384.

[165] Shirley Wu, Shiyu Zhao, Qian Huang, Kexin Huang, Michihiro Yasunaga, Kaidi Cao, Vassilis N. Ioannidis, Karthik Subbian, Jure Leskovec, and James Zou. 2024. AvaTaR: Optimizing LLM agents for tool-assisted knowledge retrieval. *arXiv e-prints* (2024), arXiv–2406.

[166] Yuwei Wu, Xuezhe Ma, and Diyi Yang. 2021. Personalized response generation via generative split memory network. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 1956–1970.

[167] Zihui Wu, Haichang Gao, Jianping He, and Ping Wang. 2025. The dark side of function calling: Pathways to jailbreaking large language models. In *Proceedings of the 31st International Conference on Computational Linguistics*. 584–592.

[168] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. 2024. OS-ATLAS: A foundation action model for generalist GUI agents. *arXiv:2410.23218* (2024).

[169] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.

[170] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2023. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv:2310.04408* (2023).

[171] Qiancheng Xu, Yongqi Li, Heming Xia, and Wenjie Li. 2024. Enhancing tool retrieval with iterative feedback from large language models. *arXiv:2406.17465* (2024).

[172] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. 2024. A survey on knowledge distillation of large language models. *arXiv:2402.13116* (2024).

[173] Yifan Xu, Xiao Liu, Xueqiao Sun, Siyi Cheng, Hao Yu, Hanyu Lai, Shudan Zhang, Dan Zhang, Jie Tang, and Yuxiao Dong. 2025. Androidlab: Training and systematic benchmarking of android autonomous agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* 1 (2025), 2144–2166.

[174] Chuanpeng Yang, Yao Zhu, Wang Lu, Yidong Wang, Qian Chen, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. 2025. Survey on knowledge distillation for large language models: Methods, evaluation, and application. *ACM Transactions on Intelligent Systems and Technology* 16, 6 (2025), 1–27.

[175] Lingfeng Yang, Zhenyuan Chen, Xiang Li, Peiyang Jia, Liangqu Long, and Jian Yang. 2024. Agent-based video trimming. *arXiv:2412.09513* (2024).
[176] Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. 2023. Gpt4tools: Teaching large language model to use tools via self-instruction. In *Advances in Neural Information Processing Systems* 36 (2023), 71995–72007.
[177] Sherry Yang, Ofir Nachum, Yilun Du, Jason Wei, Pieter Abbeel, and Dale Schuurmans. 2023. Foundation models for decision making: Problems, methods, and opportunities. *arXiv:2303.04129* (2023).
[178] Zongxin Yang, Guikun Chen, Xiaodi Li, Wenguan Wang, and Yi Yang. 2024. DoraemonGPT: Toward Understanding Dynamic Scenes with Large Language Models (Exemplified as A Video Agent). In *International Conference on Machine Learning (PMLR'24)*. 55976–55997.
[179] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv:2210.03629* (2022).
[180] Junjie Ye, Guanyu Li, Songyang Gao, Caishuang Huang, Yilong Wu, Sixian Li, Xiaoran Fan, Shihan Dou, Qi Zhang, Tao Gui, et al. 2025. Tooleyes: Fine-grained evaluation for tool learning capabilities of large language models in real-world scenarios. 156–187.
[181] Junjie Ye, Yilong Wu, Songyang Gao, Caishuang Huang, Sixian Li, Guanyu Li, Xiaoran Fan, Qi Zhang, Tao Gui, and Xuan-Jing Huang. 2024. Rotbench: A multi-level benchmark for evaluating the robustness of large language models in tool learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 313–333.
[182] Botao Yu, Frazier N. Baker, Ziru Chen, Garrett Herb, Boyu Gou, Daniel Adu-Ampratwum, Xia Ning, and Huan Sun. 2025. Tooling or not tooling? the impact of tools on language agents for chemistry problem solving. In *Findings of the Association for Computational Linguistics (NAACL'25)*. 7620–7640.
[183] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R Fung, Hao Peng, and Heng Ji. 2023. Craft: Customizing llms by creating and retrieving from specialized toolsets. *arXiv:2309.17428* (2023).
[184] Ye Yuan, Chengwu Liu, Jingyang Yuan, Gongbo Sun, Siqi Li, and Ming Zhang. 2024. A hybrid RAG system with comprehensive enhancement on complex reasoning. *arXiv:2408.05141* (2024).
[185] Aohan Zeng, Xiao Liu, Zhengxiao Du, ZihanWang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu,Wendi Zheng, Xiao Xia, et al. [n. d.]. GLM-130B: An open Bilingual Pre-trained Model. In *The Eleventh International Conference on Learning Representations*.
[186] Boyang Zhang, Yicong Tan, Yun Shen, Ahmed Salem, Michael Backes, Savvas Zannettou, and Yang Zhang. 2025. Breaking agents: Compromising autonomous LLM agents through malfunction amplification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 34952–34964.
[187] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–20.
[188] Hongming Zhang, Yan Song, Yangqiu Song, and Dong Yu. 2019. Knowledge-aware pronoun coreference resolution. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 867–876.
[189] Saizheng Zhang, Emily Dinan, Jack Urbanek, Arthur Szlam, Douwe Kiela, and Jason Weston. 2018. Personalizing dialogue agents: I have a dog, do you have pets too? *arXiv:1801.07243* (2018).
[190] Shaokun Zhang, Yi Dong, Jieyu Zhang, Jan Kautz, Bryan Catanzaro, Andrew Tao, Qingyun Wu, Zhiding Yu, and Guilin Liu. 2025. Nemotron-research-Tool-N1: Exploring tool-using language models with reinforced reasoning. *arXiv:2505.00024* (2025).
[191] Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. 2024. Agent-pro: Learning to evolve via policy-level reflection and optimization. *arXiv:2402.17574* (2024).
[192] Yinger Zhang, Hui Cai, Xierui Song, Yicheng Chen, Rui Sun, and Jing Zheng. 2024. Reverse chain: A generic-rule for llms to master multi-api planning. In *Findings of the Association for Computational Linguistics: (NAACL'24)*. 302–325.
[193] Yipeng Zhang, Xin Wang, Hong Chen, Jiapei Fan, Weigao Wen, Hui Xue, Hong Mei, and Wenwu Zhu. 2024. Large language model with curriculum reasoning for visual concept recognition. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6269–6280.
[194] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2026. Retrieval-augmented generation for ai-generated content: A survey. *Data Science and Engineering*. 1–29.
[195] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv:2303.18223* (2023).
[196] Zirui Zhao, Wee Sun Lee, and David Hsu. 2023. Large language models as commonsense knowledge for large-scale task planning. In *Advances in Neural Information Processing Systems* 36 (2023), 31967–31987.
[197] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2023. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *The Twelfth International Conference on Learning Representations*.

[198] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*

[199] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731.

[200] Wei Zhou, Mohsen Mesgar, Annemarie Friedrich, and Heike Adel. 2025. Efficient multi-agent collaboration with tool use for online planning in complex table question answering. In *Findings of the Association for Computational Linguistics (NAACL'25)*. 945–968.

[201] Yifei Zhou, Qianlan Yang, Kaixiang Lin, Min Bai, Xiong Zhou, Yu-Xiong Wang, Sergey Levine, and Li Erran Li. 2025. Proposer-agent-evaluator (pae): Autonomous skill discovery for foundation model internet agents. In *Forty-second International Conference on Machine Learning*.

[202] Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A. Rossi, Somdeb Sarkhel, and Chao Zhang. 2023. Toolchain*: Efficient action space navigation in large language models with a* search. *arXiv:2310.13227* (2023).

[203] Rui Zuo, Zifan Wang, Simon Khan, Garrett Ethan Katz, and Qinru Qiu. 2024. Why the agent made that decision: Explaining deep reinforcement learning with vision masks. *arXiv:2411.16120* (2024).

Received 22 April 2025; revised 11 November 2025; accepted 26 December 2025